

SnapATAC2: a fast, scalable and versatile tool for analysis of single-cell omics data

Kai Zhang¹, Nathan R Zemke^{1,2}, Ethan J Armand^{1,3}, and Bing Ren^{1,2,4,5}✉

¹Department of Cellular and Molecular Medicine, University of California San Diego School of Medicine, La Jolla, CA, USA

²Center for Epigenomics, University of California San Diego, La Jolla, CA, USA

³Bioinformatics and Systems Biology Program, University of California San Diego, La Jolla, CA, USA

⁴Ludwig Institute for Cancer Research, La Jolla, CA, USA

⁵Institute for Genomic Medicine, University of California San Diego, La Jolla, CA, USA

Single-cell omics technologies have ushered in a new era for the study of dynamic gene regulation in complex tissues during development and disease pathogenesis. A major computational challenge in analyzing these datasets is to project the large-scale and high dimensional data into low-dimensional space while retaining the relative relationships between cells in order to decompose the cellular heterogeneity and reconstruct cell-type-specific gene regulatory programs. Conventional dimensionality reduction methods suffer from computational inefficiency, difficulty to capture the full spectrum of cellular heterogeneity, or inability to apply across diverse molecular modalities. Here, we report a fast and nonlinear dimensionality reduction algorithm that not only more accurately captures the heterogeneities of single-cell omics data, but also features runtime and memory usage that is computationally efficient and linearly proportional to cell numbers. We implement this algorithm in a Python package named SnapATAC2, and demonstrate its superior performance, remarkable scalability and general adaptability using an array of single-cell omics data types, including single-cell ATAC-seq, single-cell RNA-seq, single-cell Hi-C, and single-cell multiomics datasets.

Single-cell omics | Dimensionality reduction | Spectral embedding

Correspondence: biren@health.ucsd.edu

Introduction

Rapid advancements in single-cell omics technologies have enabled the analysis of the gene regulatory programs encoded in the genome at unprecedented resolution and scale¹. Single-cell analysis of genomes, transcriptomes, open chromatin landscapes, histone modifications, transcription factor binding, DNA methylation, chromatin architecture, *etc.*, have provided valuable insights into the mechanisms governing cellular identity and regulation¹. However, the extreme scale and complexity of single-cell omics data often present significant computational challenges, necessitating the development of efficient, scalable, and robust methods for data analysis².

A crucial step in analyzing single-cell omics data is to project the high dimensional data onto low-dimensional space while retaining the relative relationships between cells, a process known as dimensionality reduction. This step is key to the success of downstream analyses such as clustering, batch correction, data integration, and visualization. Effective dimensionality reduction techniques are instrumental for visualization of distinct cell populations,

identification of rare cell types, and delineation of cell-type-specific transcriptional regulatory programs². Currently, single-cell omics dimensionality reduction algorithms fall into two main categories: linear and nonlinear techniques. Linear dimensionality reduction algorithms, such as principal component analysis (PCA), used by SCANPY³ and Seurat⁴, for single-cell RNA-seq (scRNA-seq) data analysis, and latent semantic indexing (LSI) used by ArchR⁵ and Signac⁶ for single-cell Assay for Transposase-Accessible Chromatin using sequencing (scATAC-seq) data analysis, are popular due to their computational efficiency and scalability.

However, these algorithms are not optimal for handling single-cell datasets with complex and nonlinear structures, such as single-cell Hi-C (scHi-C) and single-cell multimodal omics datasets. Nonlinear dimensionality reduction methods address these issues by more effectively capturing complex and often nonlinear cell relationships. Examples include latent Dirichlet allocation (LDA) used for scATAC-seq and scHi-C data^{7,8}, Laplacian-based algorithms used for scRNA-seq and scATAC-seq data⁹⁻¹³, and various neural network models developed for scRNA-seq, scATAC-seq, and scHi-C data¹⁴⁻¹⁸. Nonlinear dimensionality reduction methods have also become the standard approach for single-cell data visualization. For example, t-distributed stochastic neighbor embedding (t-SNE)¹⁹ and uniform manifold approximation and projection (UMAP)²⁰ are two widely-used algorithms for this purpose, despite recent concerns regarding their reliability and validity²¹. While nonlinear methods excel in handling complex structures and projecting data into low-dimensional manifolds, they are generally computationally inefficient, with limited scalability. For instance, LDA relies on the Markov chain Monte Carlo (MCMC) algorithm for model training, which is slow to converge, computationally expensive, and difficult to parallelize, making it difficult to be applied to large datasets²². Laplacian-based techniques like our previous work, SnapATAC⁹, necessitate computing similarity matrices between all pairs of cells, which leads to quadratic memory usage increase with the number of cells^{23,24}. Deep neural network models, known for their high training costs, often require specialized computational hardware such as GPUs to be computationally feasible.

In the present study, we describe a nonlinear dimensionality reduction algorithm that achieves both computational efficiency and accuracy in discerning cellular composition of complex tissues from a broad spectrum of single-cell omics data types. The key innovation of our algorithm is the use of a matrix-free spectral embedding algorithm to project single-cell omics data into a low-dimensional space that preserves the intrinsic geometric properties of the underlying data. Unlike conventional spectral embedding approach that requires the construction of the graph Laplacian matrix, a process that demands a storage space increasing quadratically with the number of cells, our algorithm achieves the same goal while avoiding this computationally expensive step. Specifically, we utilize the Lanczos algorithm²⁵ to derive eigenvectors while implicitly using the Laplacian matrix. This strategy significantly shortens the time and space complexity, making it linearly proportional to the number of the cells in the single-cell data. To evaluate the accuracy and utility of our algorithm, we conducted extensive benchmarking using a variety of datasets that encompass diverse experimental protocols, species, and tissue types. The results showed that our matrix-free spectral embedding algorithm outperforms existing methods in terms of speed, scalability and precision in resolving cell heterogeneity. Furthermore, we showed that our algorithm can be extended to diverse molecular

modalities of single-cell omics datasets, revealing cell heterogeneity by leveraging complementary information from different single-cell omics data types. Lastly, we developed an approach capable of performing spectral embedding on data that exceeds available computer memory, a feature that will prove beneficial for atlas-scale single-cell omics datasets.

We have implemented these algorithmic advancements in a Python package called SnapATAC2. This package is a significant revamp of the original SnapATAC, offering substantial improvements such as increased speed, reduced memory usage, more reliable performance, and a comprehensive analysis framework for diverse single-cell omics data. SnapATAC2 is freely available at <https://github.com/kaizhang/SnapATAC2>.

Results

An overview of the SnapATAC2 workflow. SnapATAC2 is a comprehensive, high-performance solution for single-cell omics data analysis. Like the original SnapATAC⁹, SnapATAC2 streamlines end-to-end scATAC-seq data analysis with an extensive set of functionality. Moreover, SnapATAC2 is designed with flexibility in mind, intended for a variety of single-cell omics data types. For instance, its dimensionality reduction subroutine is readily applicable to scATAC-seq, scRNA-seq, single-cell DNA methylation, and scHi-C data, showcasing its adaptability. To enhance performance and scalability, SnapATAC2 uses the Rust²⁶ programming language for executing computationally intensive subroutines, and provides a Python²⁷ interface for seamless installation and user-friendly operation. This combination allows for efficient processing of large-scale single-cell omics data while maintaining accessibility for researchers across various levels of expertise. To further improve scalability when handling large-scale single-cell data, on-disk data structures and out-of-core algorithms are employed whenever possible. These modifications facilitate the analysis of large datasets without overburdening system resources. Additionally, SnapATAC2 is modular and adaptable, and allows users to tailor their analysis to specific requirements or integrate it with other software packages from the scverse²⁸ ecosystem, such as SCANPY³ and scvi-tools¹⁴.

SnapATAC2 consists of four primary modules: preprocessing, embedding/clustering, functional enrichment analysis, and multi-modal omics analysis (Fig. 1a). Each module has been thoroughly revamped from the original SnapATAC, resulting in up to 100 times faster performance. The preprocessing module manages raw BAM files, calculates quality control metrics, generates cell-by-bin count matrices, and identifies doublets. These steps establish a robust foundation for subsequent analyses, ensuring data quality and integrity. At the heart of the SnapATAC2 package is the embedding/clustering module, featuring a novel matrix-free spectral embedding algorithm for dimensionality reduction, developed specifically in this study. This module incorporates dimensionality reduction, batch correction, and graph-based clustering, which are vital for distinguishing distinct cell populations and unveiling underlying biological structures. Functional enrichment analysis delves into various aspects of comprehensive data interpretation, including differential accessibility analysis and motif enrichment analysis. Lastly, the multi-modal analysis module enables the analysis of single-cell multimodal omics datasets, featuring more robust data integration, joint embedding of multiomic datasets, and the construction of gene regulatory networks utilizing

information from multiple modalities.

Matrix-free spectral embedding enables efficient and accurate dimensionality reduction of single-cell ATAC-seq data. Spectral embedding, also known as Laplacian eigenmaps, is a widely used technique for nonlinear dimensionality reduction²⁹. This method boasts several key advantages, such as locality preservation, noise reduction, and a natural connection to clustering²⁹. Spectral embedding techniques leverage the spectrum (eigenvalues and eigenvectors) of the cell similarity matrix calculated from single-cell omics datasets to perform dimensionality reduction. However, the computation of this matrix is a rate-limiting step and a memory bottleneck, creating challenges for handling datasets consisting of large numbers of cells. For example, the memory usage of the similarity matrix for a dataset with one million cells is approximately seven terabytes, far beyond the capacity of most computational servers. To address this barrier, we devise a matrix-free spectral embedding algorithm that efficiently computes eigenvectors using the Lanczos algorithm²⁵, eliminating the need for constructing a full similarity matrix (Fig. 1b, see Methods for details). This method exhibits linear space and time usage relative to the input matrix size, resulting in a faster and memory-efficient approach for processing of large datasets. Notably, our algorithm avoids heuristic approximations, delivering precise solutions, distinguishing it from previous methods that generate approximate outcomes^{10,11,30} (see Methods for details).

To benchmark the performance of SnapATAC2, we generated synthetic scATAC-seq datasets with varying cell numbers and compared the scalability of the matrix-free spectral embedding algorithm to other widely-used dimensionality reduction algorithms, such as LSI (employed by ArchR⁵ and Signac⁶), LDA (used by cisTopic⁷), and classic spectral embedding with the Jaccard index (implemented in the original SnapATAC⁹ package). We excluded neural network-based methods from this comparison due to their significantly longer runtime and the requirement for specialized computer hardware. For example, training the scBasset¹⁶ neural network model on a dataset with 10,000 cells using 16 CPU cores takes roughly 10 minutes per epoch. With 500 epochs needed for training, the entire process would require 83 hours. In contrast, SnapATAC2 and ArchR can complete the analysis in under three minutes. As shown in Fig. 1c, the runtime of SnapATAC2's matrix-free spectral embedding algorithm scales linearly with the number of cells, rivaling the fastest algorithm, LSI, which utilizes truncated singular value decomposition (SVD) for dimensionality reduction. In contrast, the original SnapATAC package experienced out-of-memory errors when processing datasets with over 40,000 cells on a Linux server equipped with 120 gigabytes of memory. Although cisTopic did not face memory constraints, it required approximately 2 hours to complete the analysis of 10,000 cells while SnapATAC2 finished in under 3 minutes. Notably, the SnapATAC2 package consumes the least amount of memory among all tested methods.

One of the objectives of SnapATAC2 is to offer comprehensive end-to-end analysis for scATAC-seq data. ArchR was previously considered the most scalable and complete software package for this purpose³¹. We therefore carried out a thorough comparison between SnapATAC2 and ArchR across eight stages in scATAC-seq analysis, including BAM file filtering and processing, data import, quality control metric computation, cell-by-bin matrix generation,

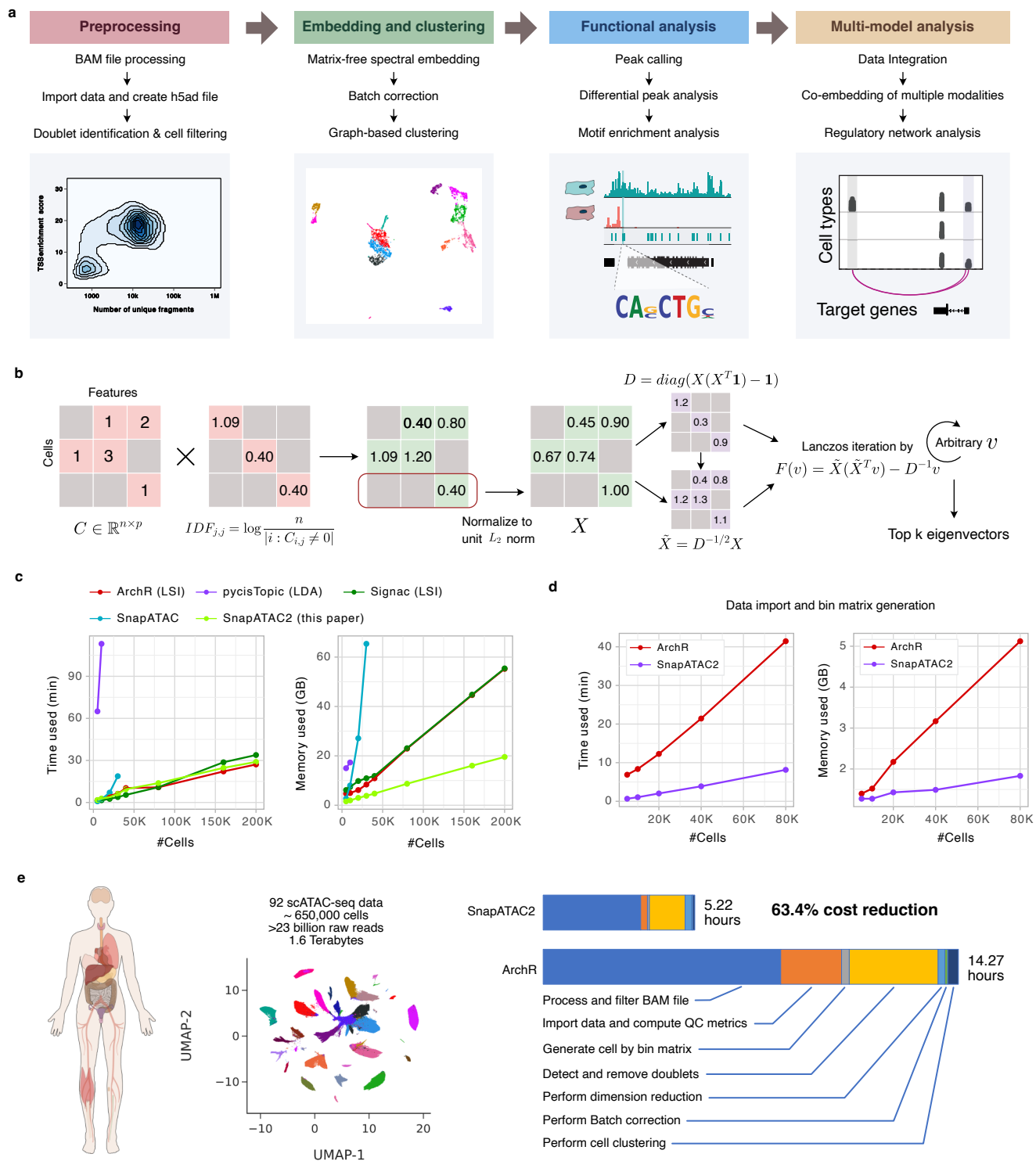


Figure 1 | SnapATAC2 enables comprehensive and scalable end-to-end analysis of single-cell omics data. **a**, Overview of the SnapATAC2 Python package, featuring four primary modules: preprocessing, embedding/clustering, functional enrichment analysis, and multi-modal analysis. **b**, Schematic representation of the matrix-free spectral embedding algorithm in SnapATAC2, consisting of four main steps: feature scaling with inverse term frequency, row-wise L_2 norm normalization, normalization using the degree matrix, and eigenvector calculation through the Lanczos algorithm²⁵. **c**, Line plots comparing running times (left) and memory usage (right) of various dimensionality reduction algorithms for scATAC-seq data. Each data point indicates the average of three independent measurements. The benchmark was conducted on a Linux server with 4 cores and 120GB memory. See **Extended Data Table 1** for additional information. **d**, Line plots comparing running times (left) and memory usage (right) of SnapATAC2 and ArchR for preprocessing fragment files in scATAC-seq analysis. The benchmarked steps include fragment file processing, quality control metric computation, and cell-by-bin matrix generation. Each data point represents the average of three independent measurements. The benchmark was conducted on a standard Linux server with 4 cores and 120GB memory. See **Extended Data Table 2** for further details. **e**, Runtime comparison between ArchR and SnapATAC2 for end-to-end analysis of 92 raw bam files produced by scATAC-seq experiments. The benchmark was performed on a Linux server with 8 cores and 64GB memory.

doublet removal, dimensionality reduction, batch correction, and clustering. In Fig. 1d, we showed that SnapATAC2 significantly outpaces ArchR in preprocessing steps while maintaining sub-linear memory usage relative to the total number of cells analyzed. Subsequently, we conducted an end-to-end analysis using both SnapATAC2 and ArchR on our previously published human single-cell atlas of chromatin accessibility³². This atlas comprises 92 scATAC-seq samples, roughly 650,000 cells, over 23 billion raw reads, and has a total size of 1.6 terabytes. As shown in Fig. 1e, SnapATAC2 completed the entire process in 5.22 hours, whereas ArchR took 14.27 hours. At this scale, SnapATAC2 is almost three times faster than ArchR, decreasing computational costs by approximately 63.4%.

SnapATAC2 produces the highest-quality cell embeddings among existing methods. We proceeded to assess the precision of our dimensionality reduction algorithm in identifying the relationships between cells, in comparison to other existing methods. For this purpose, we utilized a previously published benchmark dataset of synthetic scATAC-seq data²², consisting of ten simulated datasets with varying sequencing depths and noise levels, along with ground-truth cell type labels (Fig. 2a). To increase the difficulty of this task, we combined all ten datasets into a single dataset. We expanded our comparison to include deep neural network-based algorithms including PeakVI¹⁵ and scBasset¹⁶, as they have been reported to surpass traditional dimensionality reduction methods. We also included EpiScanpy³³, a method that utilizes PCA for dimensionality reduction. In total, we benchmarked eight methods on the merged dataset, as shown in Fig. 2b. After dimensionality reduction, we applied graph-based clustering using the Leiden algorithm³⁴ and assessed the clustering quality with the adjusted Rand index (ARI) and adjusted mutual information (AMI). ARI and AMI are both measures to compare the similarity between two data clusterings and have been routinely employed to assess the performance of clustering algorithms^{22,35}. We hypothesized that high-quality embeddings should yield clusters consistent with the ground-truth cell type labels, and hence resulted in high ARI and AMI scores. In Fig. 2b,c, we showed that SnapATAC2 and the original SnapATAC produced the highest ARI and AMI scores, followed by scBasset and LDA (cisTopic). On the contrary, ArchR and PeakVI generated the poorest embeddings, with significantly lower ARI and AMI scores than other methods, likely because these methods were not designed for handling substantial noise levels that were purposely introduced in this benchmark. These results show that the original SnapATAC and our newly developed SnapATAC2 are the only methods that are robust to large noise levels and produce high-quality embeddings.

To assess SnapATAC2's performance under conditions that mimic data generated from real experiments, we utilized a previously published scATAC-seq dataset from the human hematopoietic system³⁶. This dataset comprises 2,034 hematopoietic cells that were profiled and sorted using fluorescence-activated cell sorting (FACS) from ten cell populations, including hematopoietic stem cells (HSCs), multipotent progenitors (MPPs), lymphoid-primed multipotent progenitors (LMPPs), common myeloid progenitors (CMPs), granulocyte-macrophage progenitors (GMPs), GMP-like cells, megakaryocyte-erythroid progenitors (MEPs), common lymphoid progenitors (CLPs), monocytes (mono), and plasmacytoid dendritic cells (pDCs) (Fig. 3a). SnapATAC2 again achieved the highest ARI and AMI scores, followed by other nonlinear methods including cisTopic, PeakVI and scBasset (Fig. 3b,c). Overall, these two

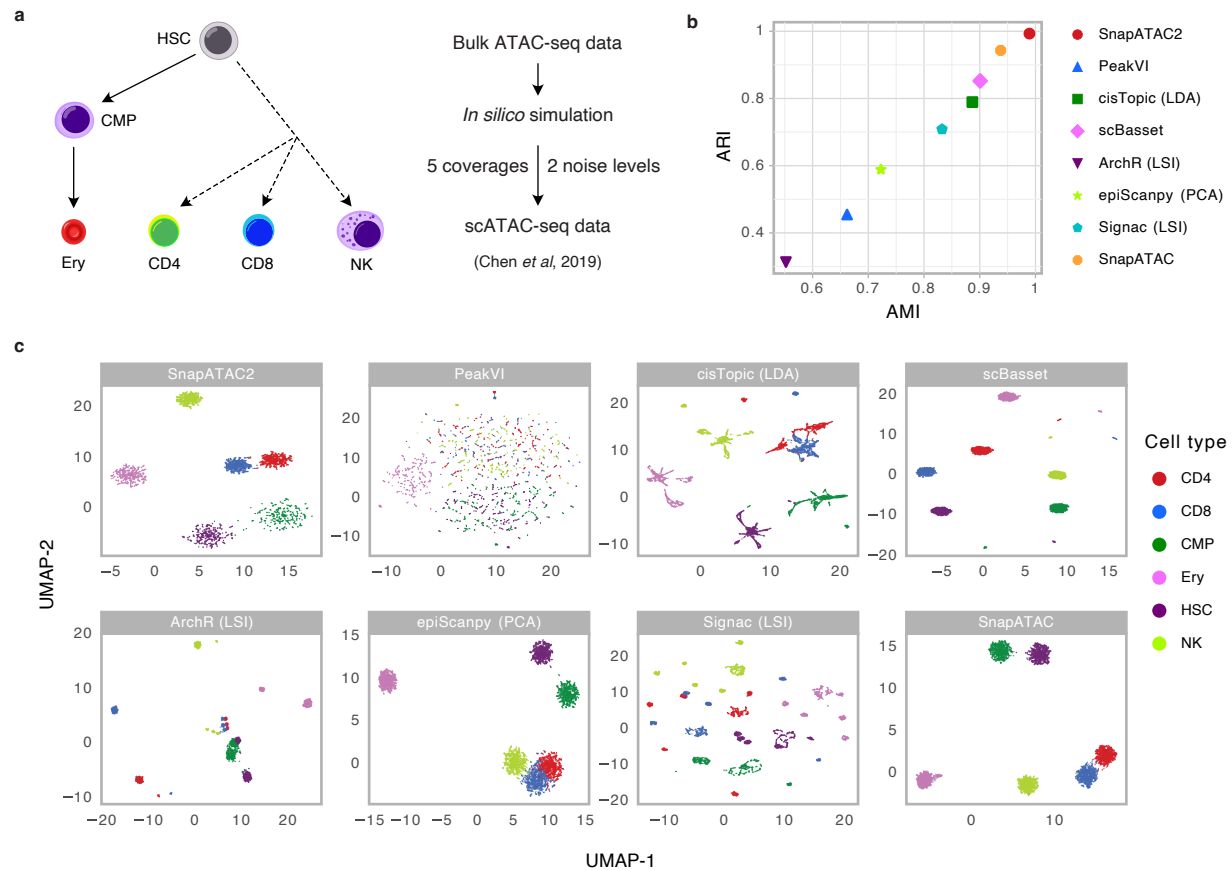


Figure 2 | Benchmarking of SnapATAC2 and other dimensionality reduction algorithms using synthetic scATAC-seq data. **a**, Synthetic scATAC-seq datasets²² with different sequencing depths (5000, 2500, 1000, 500, and 250 fragments per cell) and noise levels (0, 0.2, and 0.4) were merged into a single dataset. The merged dataset contains 9,600 cells and includes the following six cell types: hematopoietic stem cells (HSCs), common myeloid progenitor cells (CMPs), erythroid cells (Ery), natural killer cells (NK), CD4⁺ T cells (CD4), and CD8⁺ T cells (CD8). **b**, Dot plot comparing the Adjusted Rand Index (ARI, Y-axis) and adjusted mutual information (AMI, X-axis) for eight dimensionality reduction methods. Refer to **Extended Data Table 3** and **Fig. S1** for further details and additional method comparisons. **c**, UMAP visualization of the embeddings generated by each method for the simulated dataset. Individual cells are color-coded based on the cell type labels indicated in **a**.

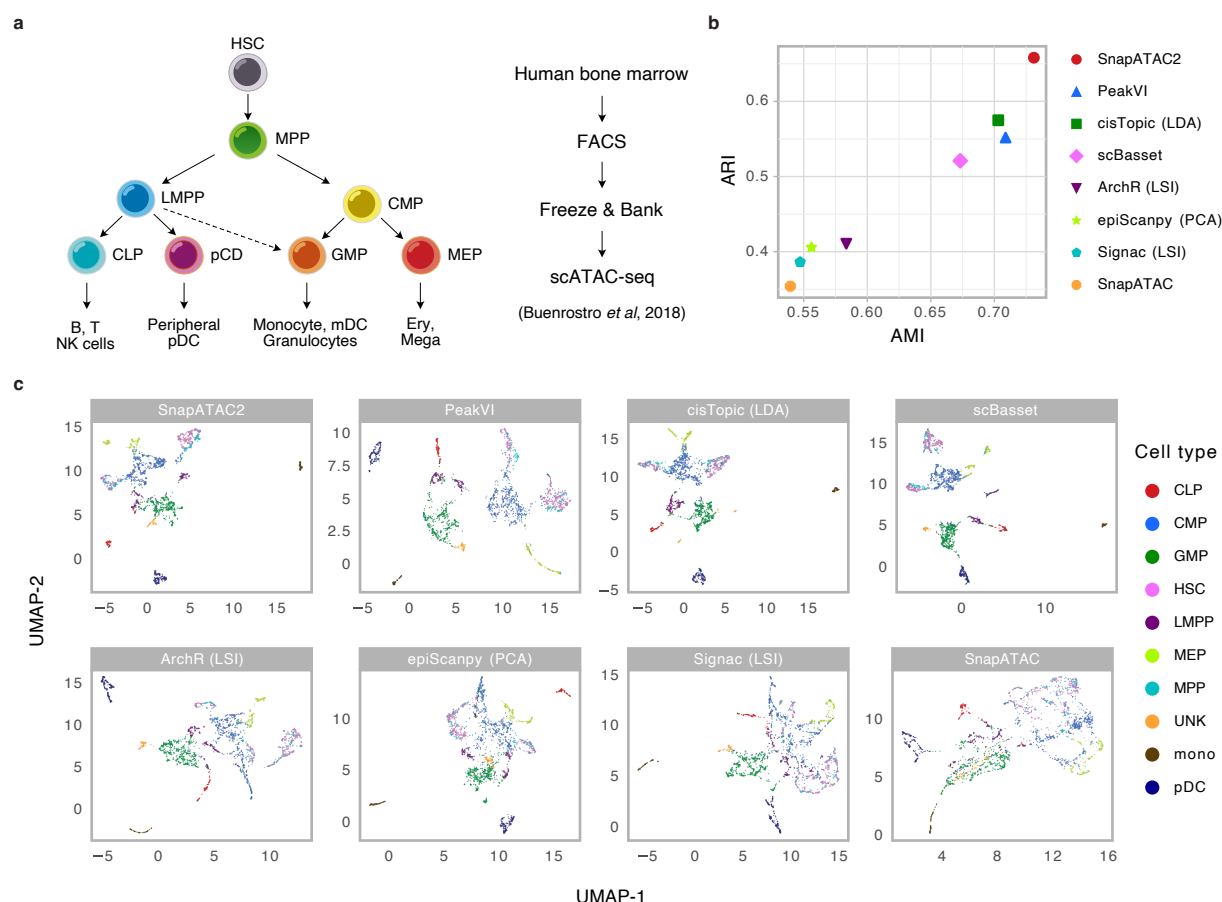


Figure 3 | Benchmarking of SnapATAC2 and other dimensionality reduction algorithms using actual scATAC-seq data with ground-truth labels. **a**, Overview of cell types analyzed in the Buenrostro2018 scATAC-seq dataset, which includes 2,034 hematopoietic cells profiled and FACS-sorted from ten cell populations: hematopoietic stem cells (HSCs), multipotent progenitors (MPPs), lymphoid-primed multipotent progenitors (LMPPs), common myeloid progenitors (CMPs), granulocyte-macrophage progenitors (GMPs), GMP-like cells, megakaryocyte-erythroid progenitors (MEPs), common lymphoid progenitors (CLPs), monocytes (mono), and plasmacytoid dendritic cells (pDCs). **b**, Dot plot comparing the Adjusted Rand Index (ARI, Y-axis) and adjusted mutual information (AMI, X-axis) for eight dimensionality reduction methods. Refer to **Extended Data Table 3** and **Fig. S2** for further details and additional method comparisons. **c**, UMAP visualization of the embeddings generated by each method for the Buenrostro2018 dataset. Individual cells are color-coded based on the cell type labels indicated in **a**.

benchmarking experiments demonstrate that nonlinear dimensionality reduction algorithms generally outperform linear methods, and SnapATAC2 produces the highest-quality cell embeddings among all methods tested.

Although SnapATAC2 achieved top performance in both benchmarking experiments, the two benchmark datasets have limited scope and may not fully represent the diversity of scATAC-seq data. Consequently, we examined additional public scATAC-seq datasets generated using different technologies, species, and tissue types^{37–42} (Table 1). Acknowledging the potential variations in cell type label quality between datasets, we restricted our analysis to broadly cited datasets. Additionally, we included six datasets where cells were annotated by an independent modality, rather than the ATAC modality, to minimize potential biases from the algorithm used to generate the cell type labels. SnapATAC2 achieved the highest AMI scores in six of the nine tested datasets and ranked among the top three methods in the remaining datasets (Fig. 4a). ARI scores exhibited similar trends, with SnapATAC2 achieving the highest scores in five of the nine datasets (Fig. 4b). In general, nonlinear dimensionality reduction methods, including SnapATAC2, PeakVI, scBasset, and cisTopic, outperformed linear methods such as EpiScanpy, ArchR, and Signac in resolving cellular heterogeneity in scATAC-seq data. The median rank of SnapATAC2 across all eleven

Table 1 | Curated scATAC-seq benchmark datasets used in the present study.

Dataset	Protocol	Tissue	Species	No. of cells	No. of cell types
10× brain 5k	10× ATAC-seq	Cortex	Mouse	2,317	10
10× PBMC 10k	10× Multiome	PBMC	Human	9,631	19
Chen ³⁸	SNARE-seq	Cerebral cortex	Mouse	9,190	22
GSE194122 ⁴¹	10× Multiome	PBMC	Human	9,876	19
Ma ³⁹	SHARE-seq	Skin	Mouse	32,231	22
Trevino ³⁷	10× ATAC-seq	Cerebral cortex	Human	8,981	13
Yao ⁴⁰	sci-ATAC-seq	Primary motor cortex	Mouse	54,844	11
Zemke_human ⁴²	10× Multiome	Primary motor cortex	Human	15,284	20
Zemke_mouse ⁴²	10× Multiome	Primary motor cortex	Mouse	45,089	19

datasets was 1, followed by PeakVI and scBasset, both with a median rank of 3 (Fig. 4c,d). Beyond its superior accuracy in identifying cell types, SnapATAC2 offers key advantages over deep neural network-based methods. It can operate without specialized hardware, consume significantly less runtime, maintain consistent performance across various datasets, and does not require extensive hyperparameter tuning.

Application of matrix-free spectral embedding to multiple single-cell omics data types. Spectral embedding has proven to be a versatile and effective technique across a broad spectrum of applications. We next explored whether this algorithm could be applied to other single-cell data types, such as scRNA-seq and scHi-C.

scHi-C data is notably sparse and exhibits an extraordinarily high dimensionality. Current computational methods struggle to fully utilize sparse scHi-C data for analyzing cell-to-cell variability in 3D genome features. Therefore, we initially focused on scHi-C data and tested our method, SnapATAC2, on two datasets with multiple cell types or known cell state information, including a sci-Hi-C dataset⁸ made public by the 4D Nucleome Project (4DN) and a dataset from Lee *et al.*⁴³. We converted scHi-C data into a cell-by-feature count matrix by flattening the contact matrices of individual cells into vectors. This count matrix served as input for SnapATAC2's matrix-free spectral embedding algorithm. The resulting cell embeddings exhibited clear patterns corresponding to the underlying cell types and cellular states (Fig. 5a,b, Fig. S13). We compared the quality of cell embeddings generated by SnapATAC2 with two existing methods: Higashi¹⁸ and scHiCluster⁴⁴. SnapATAC2 significantly outperformed scHiCluster on both datasets and demonstrated comparable performance to Higashi, the current state-of-the-art method for scHi-C analysis. In the 4DN sci-Hi-C dataset, SnapATAC2 slightly outperformed Higashi, with an ARI score of 0.925 compared to 0.874 (Fig. 5a,b). In the Lee *et al.* dataset, Higashi (ARI: 0.721) slightly surpassed SnapATAC2 (ARI: 0.639) in identifying specific cell populations (Fig. S13). However, compared to Higashi, SnapATAC2 offers a significantly reduced runtime (Extended Data Table 4) and does not require specialized hardware, making it a more practical solution for analyzing large-scale scHi-C datasets.

We also applied SnapATAC2 to scRNA-seq datasets. PCA is commonly used in scRNA-seq analysis pipelines, such as Seurat⁴ and SCANPY³, and has been shown to be highly effective in identifying cell types^{35,45}. In four out of five scRNA-seq benchmark datasets³⁵, SnapATAC2 outperformed PCA, yielding embeddings more consistent with the underlying cell types (Fig. 5c,d and Fig. S14). Overall, SnapATAC2 slightly surpassed PCA, achieving

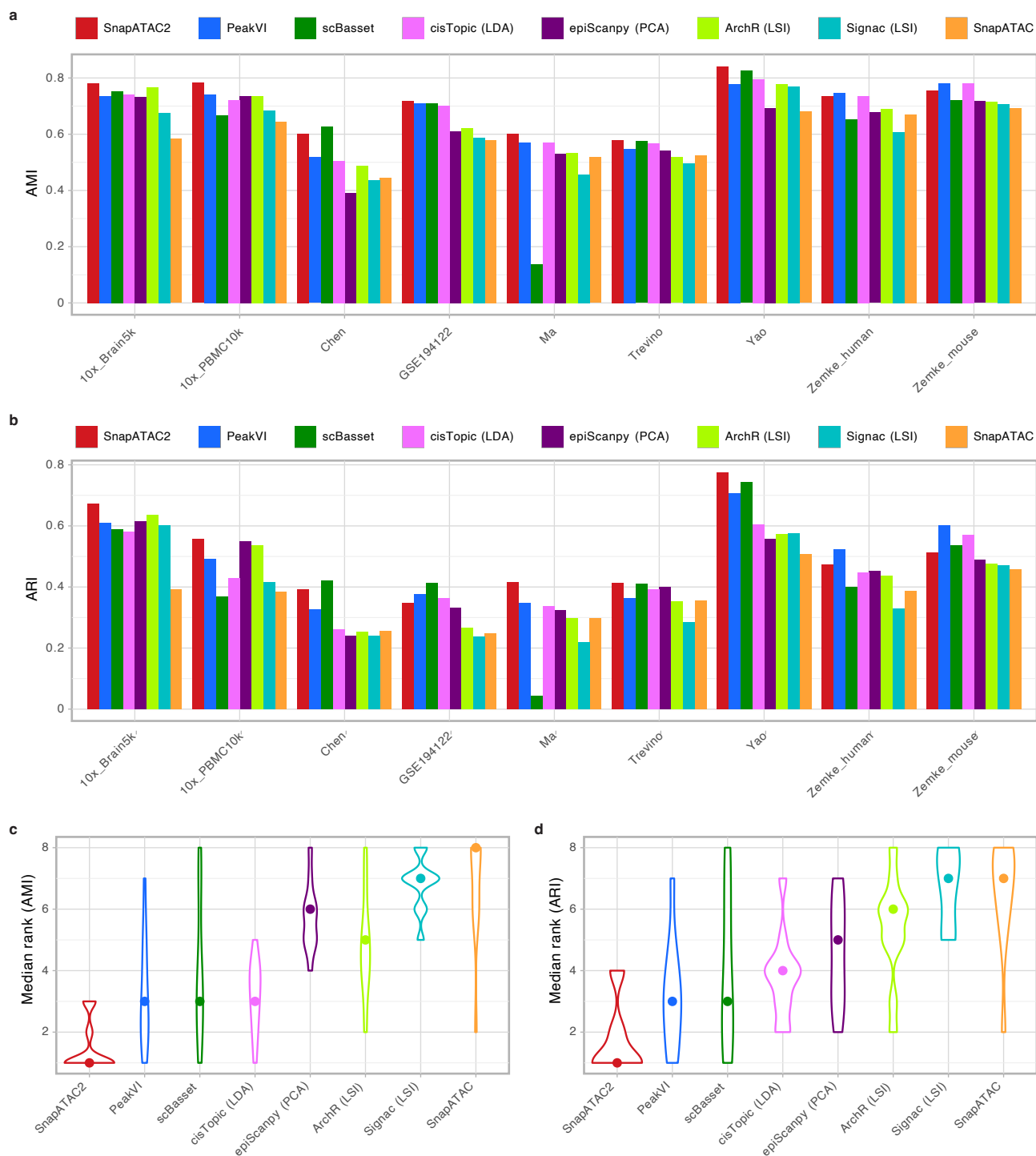


Figure 4 | SnapATAC2 demonstrates superior performance over other methods on a wide range of datasets. a,b, Bar plots illustrating the AMI (a) and ARI (b) scores of eight methods on nine previously annotated scATAC-seq datasets. For more information, refer to **Extended Data Table 3** and **Figs. S3-S12**. **c,d,** Violin plots showing the distribution of AMI (c) and ARI (d) scores for each method across all eleven scATAC-seq benchmark datasets. The median rank of each method is indicated by the dot in the violin plot.

an average ARI score of 0.900, while PCA scored 0.866. Notably, SnapATAC2 does not require data centering, a common preprocessing step in PCA analysis. When running PCA on uncentered data, the average ARI dropped to 0.841 (Extended Data Table 5), indicating that the centering step is crucial for optimal PCA performance. However, centering effectively transforms a sparse matrix into a dense matrix, which can be computationally expensive and prohibitive for large-scale datasets.

SnapATAC2's dimensionality reduction algorithm is also applicable to single-cell DNA methylation data. When applied to a 5-methylcytosine sequencing 2 (snmC-seq2) data generated in mouse pituitaries⁴⁶, SnapATAC2 produced cell embeddings that are largely consistent with the cell types identified by the original study (Fig. S15). Notably, the method provided finer resolution for some cell types, such as somatotropes and lactotropes.

In conclusion, SnapATAC2 is a versatile and effective method for the analysis of various single-cell data types, including scATAC-seq, scHi-C, scRNA-seq, and single-cell DNA methylation data. It demonstrates comparable or superior performance to existing methods, while offering practical advantages such as reduced runtime and no need for specialized hardware.

SnapATAC2 enables robust joint embedding of single-cell multiomics data. The rapid expansion of single-cell multimodal omics technologies, such as 10× multiome (ATAC/RNA-seq), Paired-Tag⁴⁷, and single-cell Methyl-HiC/snm3C-seq⁴³, has provided powerful tools for investigating gene regulatory mechanisms. However, current methods for analyzing these data often focus on a single modality, underutilizing the full potential of multimodal datasets. We therefore investigated the applicability of our algorithm to single-cell multimodal omics data. Multi-view spectral embedding is an extension of spectral embedding, which enables the joint embedding of multiple data representation views. This method has demonstrated its ability to harness complementary information from individual views and enhance performance in downstream analyses, making it an ideal candidate for analyzing single-cell multiomics data. The multi-view spectral embedding process typically consists of three steps: first, a similarity or kernel matrix is calculated from each view; second, a joint kernel matrix is constructed by combining or co-regularizing the kernel matrices in a certain manner; and lastly, spectral embedding is performed using the joint kernel matrix. In this study, we opted for kernel addition to combine the kernel matrices, as it has proven to be an effective method for achieving excellent clustering results^{47,48}. Moreover, kernel addition enables the extension of the matrix-free spectral embedding algorithm to multi-view spectral embedding while maintaining the linear time and space complexity of the algorithm (see Methods for details).

We applied this matrix-free multi-view spectral embedding algorithm to a 10× Genomics multiome dataset, which jointly profiles chromatin accessibility and the transcriptome for approximately 10,000 human peripheral blood mononuclear cells (PBMCs). To better evaluate the algorithm's performance, we first annotated the cells according to a previously published single-cell atlas of human PBMCs⁴. To compare the performance of joint embedding with individual views, we also performed spectral embedding on each modality separately. Our findings revealed that, while independent unsupervised analyses of RNA and ATAC data generated predominantly consis-

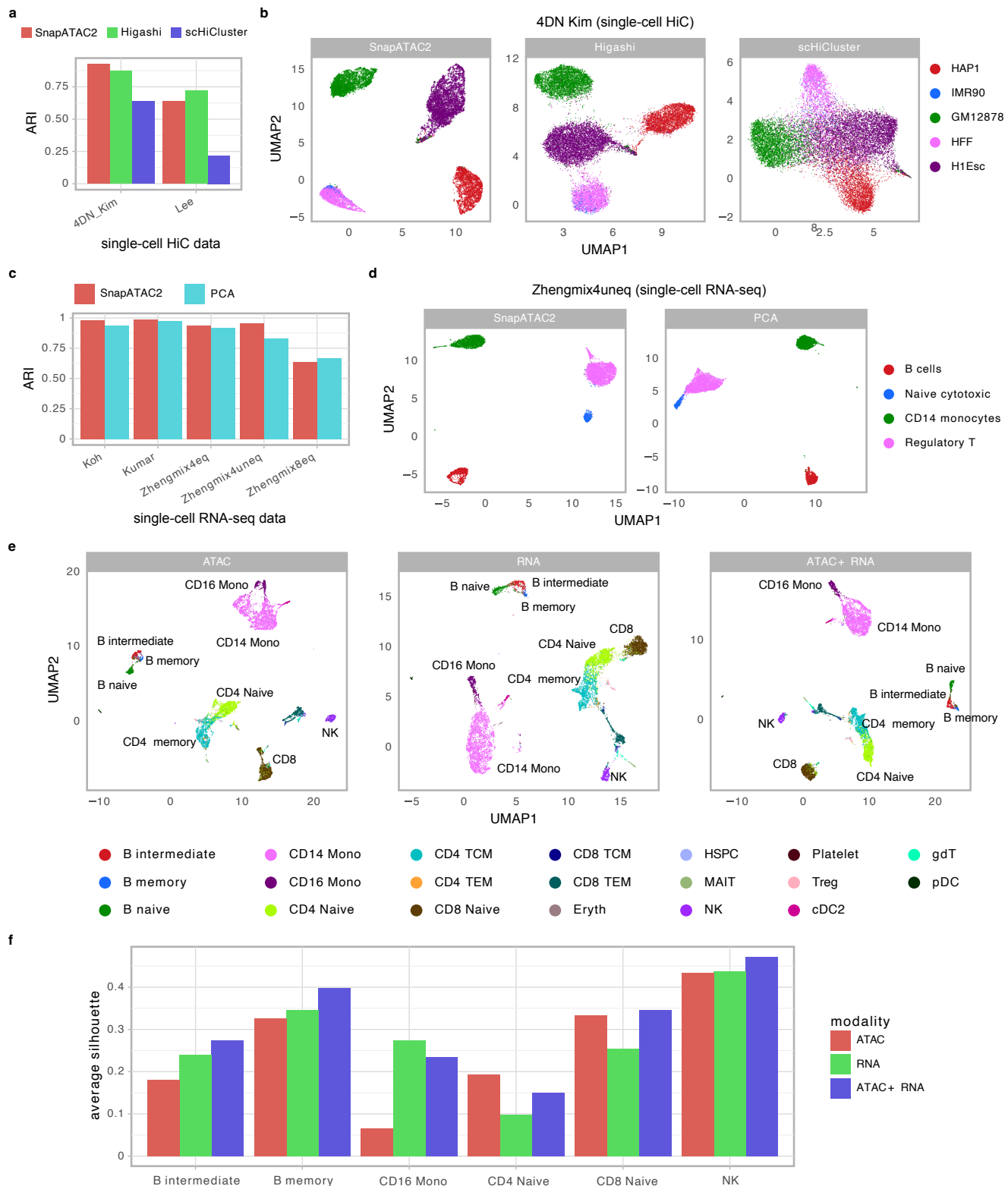


Figure 5 | SnapATAC2 enables analysis of diverse single-cell omics data types. **a**, Bar plot comparing SnapATAC2, Higashi, and scHiCluster in terms of clustering performance for scHi-C data. For more details, see **Extended Data Table 4**. **b**, UMAP visualization of the embeddings generated by SnapATAC2, Higashi, and scHiCluster for the 4DN Kim dataset⁸. Cells are color-coded based on cell type labels. **c**, Bar plot comparing SnapATAC2 and PCA (implemented in SCANPY) for clustering scRNA-seq datasets³⁵. For more details, see **Extended Data Table 5**. **d**, UMAP visualization of the embeddings produced by SnapATAC2 and PCA (SCANPY) for the Zhengmix4uneq dataset³⁵. Cells are color-coded according to cell type labels. **e**, UMAP visualization of the embeddings generated by SnapATAC2 using ATAC modality (left panel), RNA modality (middle panel), or both modalities (right panel). Cells are color-coded based on cell type labels. **f**, Bar plot comparing the average silhouette scores of selected cell types derived from embeddings produced by ATAC modality, RNA modality, or both modalities.

tent cell classifications, there were notable differences present (Fig. 5e,f). For instance, CD8⁺ and CD4⁺ T cells were close to each other when analyzing the transcriptome, but separated clearly in the ATAC data (Fig. 5e,f). Conversely, intermediate and memory B cells partially overlapped when analyzing the ATAC data but were more distinguishable in the transcriptomic data (Fig. 5e,f). In comparison to the separate analysis of either modality, multi-view spectral embedding using both modalities clearly separated CD4⁺ and CD8⁺ T cells and uncovered subtle heterogeneity within B cells. Overall, the joint embedding of ATAC and RNA data enhanced the separation of cell types and revealed subtle heterogeneity within cell types, as evidenced by the increased silhouette scores across different cell types (Fig. 5f).

SnapATAC2 overcomes memory constraints for analyzing atlas-scale datasets. SnapATAC2's matrix-free spectral embedding algorithm features linear space and time complexity, positioning it among the top-performing solutions. With 120 gigabytes of memory, SnapATAC2 can effectively perform dimensionality reduction for millions of cells. However, memory may still be a limiting factor when processing atlas-scale datasets, especially when combining data from multiple consortium studies or analyzing those with extremely high sequencing depth. To address this limitation, we developed an algorithm that circumvents the memory bottleneck of a computer, enabling the processing of massive numbers of cells in a reasonable timeframe (Fig. 6a). Our approach involved creating an on-disk data structure for efficient storage and access to portions of the count matrix without loading the entire matrix into memory. We then performed matrix-free spectral embedding on a subsampled count matrix to obtain a reference embedding. Using this reference embedding, we applied the Nyström algorithm⁴⁹ to calculate the embedding for all cells without needing to ever store the complete count matrix in memory. Additionally, the final process can be distributed across multiple machines for a high degree of parallelism, enabling the processing of billions of cells relatively quickly. This feature is essential for analyzing large amounts of single-cell data generated from consortium efforts such as the BRAIN Initiative Cell Atlas Network (BICAN), the Human Cell Atlas (HCA), and the Human BioMolecular Atlas Program (HuBMAP).

Subsampling and approximation inherently lead to a reduction in accuracy. To evaluate this effect, we compared the embeddings generated from a subsampled dataset to those produced by the complete dataset. We discovered that our out-of-core algorithm yielded embeddings that were largely consistent with those generated by the complete dataset at various subsampling rates (Fig. 6b and Extended Data Table 6). For comparison, we conducted the same analysis using the subsampled LSI algorithm implemented in ArchR. Our algorithm demonstrated greater accuracy across different subsampling rates compared to subsampled LSI (Fig. 6b). For instance, at a 10% sampling rate, the average ARI score across nine experiments for the embeddings generated by our algorithm was nearly 0.7, while the average ARI score for those generated by subsampled LSI was about 0.5 (Fig. 6b). This finding suggests that SnapATAC2 is more robust to subsampling approaches necessary for analyzing large-scale datasets.

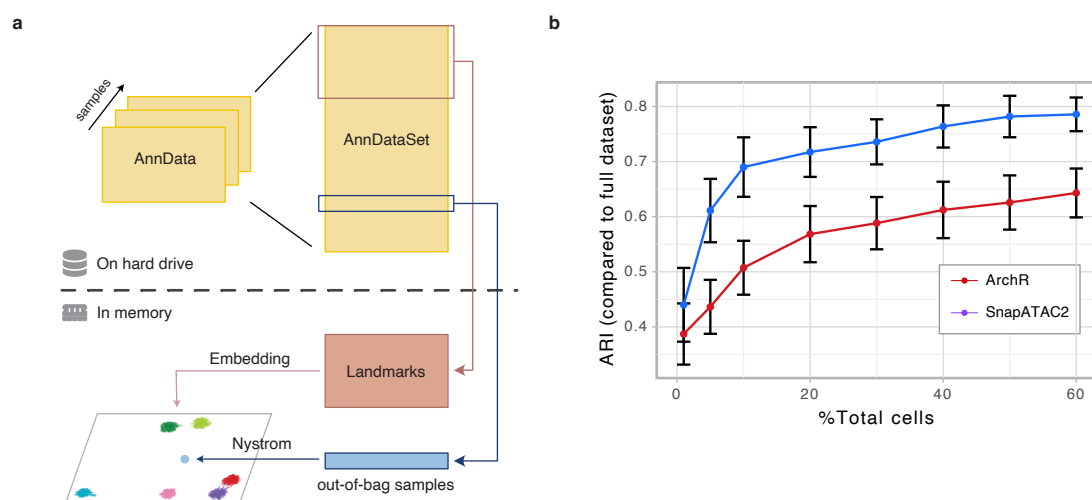


Figure 6 | SnapATAC2 overcomes memory constraints for analyzing atlas-scale datasets. a, Schematic representation of the out-of-core algorithm implemented in SnapATAC2 for processing datasets that surpass available memory capacity. b, Line plot comparing ARI scores when undergoing different levels of subsampling. ARI scores were calculated by comparing the embeddings generated from the complete dataset with those predicted using the subsampled dataset. Error bars represent standard deviation of measurements across nine independent datasets. See **Extended Data Table 6** for further details.

Discussion

In the present study, we describe a software package named SnapATAC2 for the analysis of a diverse array of single-cell omics data. The performance of SnapATAC2 exceeds that of existing dimensionality reduction methods in terms of accuracy, noise robustness, and scalability, thus providing researchers with a powerful tool for investigating gene regulatory programs using single-cell genomics, transcriptomics and epigenomics analysis.

The key innovation of SnapATAC2 lies in its matrix-free spectral embedding algorithm for dimensionality reduction. While numerous algorithms have been proposed to expedite spectral embedding^{10,11,23,30}, our algorithm stands out as it does not rely on sub-sampling or approximations, delivering the exact solution. This algorithm not only outperforms current methods in identifying cell clusters and heterogeneity but also maintains computational efficiency, making it highly suitable for large-scale single-cell omics data analysis. Furthermore, we demonstrated the versatility of the matrix-free spectral embedding algorithm by applying it to various single-cell data types, including scATAC-seq, scRNA-seq, single-cell DNA methylation and scHi-C data. Additionally, the multi-view extension of this algorithm enables the joint embedding of single-cell multiomics data and better reveals cellular heterogeneity of a mixed population, which is essential for understanding the complex dynamics of cellular states and functions. This cross-modality capability suggests that the underlying algorithm of SnapATAC2 can be adapted for analyzing different single-cell data types, potentially serving as the foundation for a unified multi-omics framework.

One limitation of the matrix-free spectral embedding algorithm is that it currently is implemented using only cosine function based similarity. For some data types, researchers may prefer to use other metrics to quantify the cell-to-cell similarity. For instance, in our findings, the Euclidean distance yielded more accurate results for the protein expression data used in Cellular Indexing of Transcriptomes and Epitopes by Sequencing (CITE-seq) experiments⁵⁰.

Future developments could extend the matrix-free algorithm to accommodate other similarity metrics. For instance, a potential solution involves leveraging a small set of landmark points to transform the given data into sparse feature vectors⁵¹, followed by the application of the scalable matrix-free spectral embedding algorithm. In conclusion, SnapATAC2 represents a substantial advancement in single-cell data analysis, offering an accessible, scalable, and high-performance solution for researchers studying epigenomics. With continued development and optimization, SnapATAC2 has the potential to become a general tool in single-cell multi-omics data analysis, ultimately facilitating novel biological discoveries.

Methods

SnapATAC2's high performance preprocessing module. SnapATAC2's preprocessing module including BAM file processing, fragment file importing, QC metrics calculation, and cell by feature count matrix generation. All functions in this module are implemented using out-of-core algorithms, which enables SnapATAC2 to handle large-scale single-cell datasets efficiently without being limited by memory constraints.

Converting BAM files to fragment files. SnapATAC2 is capable of processing unfiltered BAM files, converting them into files containing the genomic coordinates of the fragments. A fragment record consists of five specific fields: the reference genome chromosome of the fragment, the adjusted start position of the fragment on the chromosome, the adjusted end position of the fragment on the chromosome, the cell barcode associated with the fragment, and the total number of read pairs linked to the fragment. In order to convert BAM files into fragment files, the following steps are taken. First, reads that are unmapped, not primary alignment, improperly aligned, have mapping quality less than 30, fail platform/vendor quality checks, or are optical duplicates are removed. Next, using out-of-core algorithms, the remaining reads are sorted by cell barcodes, and duplicate reads for each unique cell barcode are eliminated. Finally, the filtered and deduplicated BAM records are converted into fragments. This process is implemented by the "snapatac2.pp.make_fragment_file" function within the SnapATAC2 package.

Compute QC metrics. SnapATAC2 offers a comprehensive set of QC metrics to assess data quality, which include the number of unique fragments per cell, the proportion of mitochondrial reads per cell, the proportion of duplicate reads per cell, and the TSS enrichment score per cell. To calculate the TSS enrichment score, we first retrieved the TSS positions from a gene annotation file. We then aggregated Tn5-corrected insertions within a +/- 2000 bp range relative to each unique TSS (strand-corrected) across the entire genome. Subsequently, this profile was normalized based on the mean accessibility within the +/- (1900 to 2000) bp range from the TSS and smoothed at 11 bp intervals. The maximum value of the smoothed profile was taken as the TSS enrichment. The SnapATAC2 package's "snapatac2.pp.import_data" function facilitates this process.

Compute cell by feature count matrix. SnapATAC2 provides various methods to calculate the cell-by-feature count matrix. The "snapatac2.pp.add_tile_matrix" function computes the cell-by-bin matrix by dividing the genome into

a fixed number of bins. The “snapatac2.pp.make_peak_matrix” function calculates the cell-by-peak matrix based on the peak file, while the “snapatac2.pp.make_gene_matrix” function computes the cell-by-gene matrix using the gene annotation file.

Dimensionality reduction using spectral embedding. In this section we outline the core algorithms used to perform dimensionality reduction in the SnapATAC2 package. We first describe the preprocessing steps and then the classic spectral embedding method that works for arbitrary similarity metrics. Finally, we describe the matrix-free spectral embedding algorithm that works only for cosine similarity, but substantially decreases the running time and memory usage. Note the steps described below can be accomplished using the “snapatac2.tl.spectral” function from the SnapATAC2 package.

Preprocessing. Given a cell by feature count matrix $C \in \mathbb{R}^{n \times p}$, we first scale the columns of the matrix by the Inverse Document Frequency (IDF). The IDF of a column or a feature f is defined by $idf(f) = \log \frac{n}{1 + |\{i: C_{i,f} \neq 0\}|}$.

Spectral embedding. Assuming the cell by feature count matrix C has been preprocessed according to the procedures described above, in classic spectral embedding we first compute the $n \times n$ pairwise similarity matrix W such that $W_{ij} = \delta(C_{i*}, C_{j*})$, where $\delta: \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ is the function defines the similarity between any two cells. Typical choices of δ include the Jaccard index and the cosine similarity. We then compute the symmetric normalized graph Laplacian $L_{sym} = I - D^{-1/2} W D^{-1/2}$, where I is the identity matrix and $D = \text{diag}(W \mathbf{1})$. The bottom eigenvectors of L_{sym} are selected as the lower dimensional embedding. The corresponding eigenvectors can be computed alternatively as the top eigenvectors of the similarly normalized weight matrix: $\tilde{W} = D^{-1/2} W D^{-1/2}$.

Matrix-free spectral embedding with cosine similarity. In this section, we introduce a matrix-free algorithm for spectral embedding that avoids calculating the similarity matrix. This approach is specifically designed for cosine similarity. The cosine similarity between two vectors A and B is given by $S_c(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$. To express the cosine similarity using matrix operations, we first rescale the non-negative count matrix C to obtain a new matrix X , such that the rows of X have unit L_2 norm. Consequently, the cosine similarity matrix between rows of X can be represented as XX^T .

In traditional spectral clustering algorithms, it is necessary to set the diagonals of the similarity matrix to zero⁵². This can be accomplished by subtracting the identity matrix from the similarity matrix, resulting in the final similarity matrix $W = XX^T - I$. The degree matrix can then be calculated as $D = \text{diag}((XX^T - I)\mathbf{1}) = \text{diag}(X(X^T \mathbf{1}) - \mathbf{1})$. The normalized similarity matrix, denoted as $\tilde{W}$, can then be computed as follows:

$$\tilde{W} = D^{-1/2} XX^T D^{-1/2} - D^{-1} = \tilde{X} \tilde{X}^T - D^{-1}$$

where $\tilde{X} = D^{-1/2} X$. It is important to note that $\tilde{X}$ has the same dimensions as X , and if X is sparse, $\tilde{X}$ preserves the sparsity pattern of X . Conventional spectral embedding algorithms compute $\tilde{W}$ and select its top eigenvectors

as the lower-dimensional embedding. Previous work has attempted to compute the top eigenvectors of an approximation of $\tilde{W}$ to avoid the need for computing the full similarity matrix³⁰. In other studies^{10,11}, the authors chose not to set the diagonals of the similarity matrix to zero. Consequently, the eigendecomposition of $\tilde{W}$ is equivalent to the SVD of $\tilde{X}$, which can be computed efficiently. However, our benchmarking reveals that setting the diagonal of W to zero is necessary as it significantly improve the embedding quality (Fig. S12).

Unlike previous work, we offer an exact solution to the problem. We apply the Lanczos algorithm²⁵, an iterative method for computing the top eigenvectors of a symmetric matrix, to our problem without ever calculating $\tilde{W}$. This requires computing the matrix-vector product between $\tilde{W}$ and $\mathbf{v}$ in each iteration, as follows: $\tilde{W}\mathbf{v} = \tilde{X}(\tilde{X}^T\mathbf{v}) - D^{-1}\mathbf{v}$, where $\mathbf{v}$ is current solution to the eigenvalue problem and is iteratively refined by the Lanczos algorithm. By using the specific order of operations shown in the formula, we can reduce the computational cost of the matrix-vector product to $2z + n$, where n is the number of rows in X and z is the number of non-zero elements in X . In comparison, performing this operation on the full similarity matrix requires n^2 computations, which is prohibitively expensive for large number of cells. Thus, our matrix-free method is significantly faster and more memory efficient. The pseudocode for our algorithm is shown in Fig. S16a.

Nyström method for out-of-sample embedding. The matrix-free method described above is very fast and memory efficient. However, for massive datasets with hundreds of millions of cells, storing the cell by feature count matrix C may already be a challenge. In this section, we describe an on-line embedding method that can be applied to datasets that do not fit into physical memory. The key idea here is to use the Nyström method²⁴ to construct low-rank matrix approximations.

Given a symmetric similarity matrix $W \in \mathbb{R}^{n \times n}$, the Nyström method constructs a low-rank approximation $\hat{W}$ of W by sampling $l \ll n$ columns from W . Without loss of generality, we assume that the first l columns of W are sampled. The W matrix can then be partitioned into four blocks: $W = \begin{bmatrix} A & B \\ B^T & C \end{bmatrix}$, where columns $\begin{bmatrix} A \\ B^T \end{bmatrix}$ are our samples. The low-rank approximation $\hat{W}$ is then defined as,

$$\hat{W} = \begin{bmatrix} A \\ B^T \end{bmatrix} A^{-1} \begin{bmatrix} A & B \end{bmatrix} = \begin{bmatrix} A & B \\ B^T & B^T A^{-1} B \end{bmatrix}$$

As a result, the degree matrix D can be approximated as,

$$\hat{D} = \text{diag}(\hat{W}\mathbf{1}) = \begin{bmatrix} A\mathbf{1} + B\mathbf{1} \\ B^T\mathbf{1} + B^T A^{-1} B\mathbf{1} \end{bmatrix}$$

The eigenvector of $\tilde{W}$ can then be approximated by,

$$\hat{U} = \hat{D}^{-\frac{1}{2}} \begin{bmatrix} A \\ B^T \end{bmatrix} \begin{bmatrix} A\mathbf{1} + B\mathbf{1} \end{bmatrix}^{-\frac{1}{2}}$$

The time complexity of the approach discussed above is $\mathcal{O}(nl^2 + l^3)$. The space complexity amounts to $\mathcal{O}(nl)$, as it is necessary to store the sampled columns of W . In large-scale applications, sampling even a small subset of columns can prove to be exceedingly costly. As a result, rather than computing the inverse of A , we utilize an algorithm previously described⁴⁹ to approximate the degree matrix. The full algorithm's pseudocode can be found in Fig. S16b,c.

Multi-view spectral embedding. In this section we extend our matrix-free spectral embedding method to perform dimensionality reduction on multi-modal single-cell data. Assume we have data in multiple views, *e.g.*, chromatin accessibility and gene expressions, represented by a sequence of count matrices $\{X_1, X_2, \dots, X_k\} \in \mathbb{R}^{n \times p_k}$. Our objective is to obtain a low-dimensional representation of the data while preserving cell similarity in each view using the spectral embedding method. One approach involves calculating the similarity matrix for each view, normalizing them, and subsequently summing them. The resulting matrix is then used to compute the spectral embedding. This straightforward strategy has proven effective in revealing clusters in prior research^{47,48}. However, it necessitates the computation of the similarity matrix for each view, which is computationally demanding. In this section, we present an algorithm that is both time and space-efficient for computing this embedding.

We first normalize each X_i such that the rows of X_i have unit L_2 norm. We then define X as the horizontally concatenated view of the sequence of matrices,

$$X = \left(\sqrt{\frac{\lambda_1}{\|W_1\|_F}} X_1 \quad \sqrt{\frac{\lambda_2}{\|W_2\|_F}} X_2 \quad \dots \quad \sqrt{\frac{\lambda_k}{\|W_k\|_F}} X_k \right)$$

where λ_k is the user defined weights measuring the relative importance of each view; $W_k = X_k X_k^T - I$ is the similarity matrix of the k -th view; $\|W_k\|_F$ is the Frobenius norm of W_k . We can see that,

$$\begin{aligned} X X^T - \sum_k \frac{\lambda_k}{\|W_k\|_F} I &= \sum_k \frac{\lambda_k}{\|W_k\|_F} (X_k X_k^T - I) \\ &= \sum_k \lambda_k \frac{W_k}{\|W_k\|_F} \end{aligned}$$

Without loss of generality, we can assume $\sum_k \frac{\lambda_k}{\|W_k\|_F} = 1$. In practice, this can be achieved by normalizing λ_k . The above equation can now be written as,

$$X X^T - I = \sum_k \lambda_k \frac{W_k}{\|W_k\|_F}$$

Therefore, the matrix $W = XX^T - I$ is a linear combination of the normalized similarity matrices of the individual views. To compute the spectral embedding of W , it suffices to apply the matrix-free spectral embedding method described above to the concatenated view X . This algorithm is implemented in the “snapatac2.tl.multi_spectral” function from the SnapATAC2 package. The pseudocode for this algorithm is shown in Fig. S16d.

Eigenvector selection in spectral embedding. Not all eigenvectors produced by spectral embedding are informative and relevant for clustering tasks. Selecting appropriate eigenvectors is essential, as using uninformative or irrelevant ones can lead to suboptimal clustering results. We found that the widely-used elbow method for determining the number of eigenvectors is not consistently reliable in practice. To identify relevant eigenvectors, we propose a simple heuristic based on the eigenvalues of the graph Laplacian matrix. In this approach, each eigenvector is weighted by the square root of its corresponding eigenvalue, and these weighted eigenvectors are then employed for further analyses.

Benchmarking datasets. Synthetic scATAC-seq dataset and the Buenrostro2018 dataset were downloaded from²². Additional nine scATAC-seq benchmarking datasets were listed in Table 1. The scHi-C datasets were downloaded from¹⁸. The scRNA-seq datasets were downloaded from³⁵. All benchmarking datasets are converted into a uniform format and are available at <https://osf.io/hfs2v/>.

Benchmarking algorithms. In order to ensure the reproducibility of our benchmarking results, we have prepared a separate Docker image for each method. These images can be downloaded from <https://hub.docker.com/u/kaizhang>. In this section, we provide a brief overview of each method’s usage. To access the code required for reproducing the benchmarking results, visit: <https://github.com/kaizhang/single-cell-benchmark>.

ArchR. ArchR is a R package for analyzing scATAC-seq data. To generate the lower-dimensional embedding of the data, we used the “ArchR:::computeLSI” function with the default parameters. The output dimension was set to 30. After performing the SVD, ArchR scales the singular vectors by the singular values. As a result, component selection is not necessary so we used all 30 dimensions for downstream analysis. Note ArchR includes three variants of the LSI algorithm: “TF-logIDF”, “log(TF-IDF)”, and “logTF-logIDF”. Although we have benchmarked all three variants, we only report the results for the “log(TF-IDF)” variant in the main text as it is the default setting.

Signac. Signac is a R package for analyzing scATAC-seq data. To generate the lower-dimensional embedding of the data, we used the “Signac:::RunTFIDF.default” and “Signac:::RunSVD.default” functions with the default parameters. The output dimension was set to 30. We used the elbow method to select the number of components for downstream analysis. Note Signac includes four variants of the LSI algorithm: “IDF”, “TF-logIDF”, “log(TF-IDF)”, and “logTF-logIDF”. Although we have benchmarked all four variants, we only report the results for the “log(TF-IDF)” variant in the main text as it is the default setting.

433 **EpiScanpy.** EpiScanpy is a Python package for analyzing scATAC-seq data. We first normalized the count matrix using
 434 “episcanpy.pp.normalize_per_cell” and “episcanpy.pp.log1p” functions with the default parameters. We then used
 435 the “episcanpy.pp.pca” function to generate the lower-dimensional embedding of the data. The output dimension
 436 was set to 30.

437 **SCALE.** SCALE is a Python package for performing dimensionality reduction on scATAC-seq data. We used the
 438 command “SCALE.py” with following parameters to generate the lower-dimensional embedding: “-min_peaks 0
 439 -min_cells 0 -l 30”. Additionally, as we knew the number of cell types in the benchmarking datasets, we set the “-k”
 440 parameter (the number of clusters) to the true number of cell types.

441 **PeakVI.** PeakVI is a Python package for performing dimensionality reduction on scATAC-seq data. We used the
 442 “scvi.model.PEAKVI” function to create a model with the default parameters. The dimensionality of the latent
 443 variable was set to 30.

444 **scBasset.** scBasset is a Python package for performing dimensionality reduction on scATAC-seq data. We followed
 445 the instructions in the scBasset GitHub repository to generate the lower-dimensional embedding of the data. The
 446 dimensionality of the latent variable was set to 30.

447 **pycisTopic.** pycisTopic is a Python package for analyzing scATAC-seq data. To generate the lower-dimensional em-
 448 bedding of the data, we first created a model using the “create_cistopic_object” function with the default param-
 449 eters. We then used “run_cgs_models” to train the model with the following parameters: “n_iter=300, alpha=50,
 450 alpha_by_topic=True, eta=0.1, eta_by_topic=False”. We trained six models with different dimensions of the la-
 451 tent variable: 5, 10, 15, 20, 25, and 30. We then used the “evaluate_models” function to select the best model for
 452 downstream analysis.

453 **SnapATAC.** SnapATAC is a R package for analyzing scATAC-seq data. For datasets with number of cells less than
 454 20,000, we used the “SnapATAC::runDiffusionMaps” function with the default parameters to generate the lower-
 455 dimensional embedding of the data. For datasets with number of cells more than 20,000, running “SnapATAC::runDiffusionMaps”
 456 on the full dataset requires a large amount of memory. In this case, we applied “SnapATAC::runDiffusionMaps”
 457 on a subset of the data and then used the “SnapATAC::runDiffusionMapsExtension” function to generate the lower-
 458 dimensional embedding of the full dataset. The output dimension was set to 30. We used the “SnapATAC::weightDimReduct”
 459 to scale eigenvectors by their corresponding eigenvalues. The scaled eigenvectors were then used for downstream
 460 analysis.

461 **Higashi.** Higashi is a Python package for analyzing scHi-C data. We followed the instructions in the Higashi GitHub
 462 repository to generate the lower-dimensional embedding of the data. The dimensionality of the cell embeddings
 463 was set to 30.

scHiCluster. scHiCluster is a Python package for analyzing scHi-C data. We followed the instructions in the scHi-Cluster GitHub repository to generate the lower-dimensional embedding of the data. The dimensionality of the cell embeddings was set to 30.

SCANPY. SCANPY is a Python package for analyzing scRNA-seq data. To generate the lower-dimensional embedding of the data, we first applied the “scanpy.pp.normalize_total” and “scanpy.pp.log1p” functions to preprocess the data. We then used the “scanpy.pp.highly_variable_genes” function to select highly variable genes with “n_top_genes=5000”. The data was then scaled using “scanpy.pp.scale” with “max_value=10” and inputted to the “scanpy.tl.pca” function to get lower-dimensional embedding. The output dimension was set to 30.

Benchmarking metrics. We generated a lower-dimensional embedding for each method and used it to create a k -nearest neighbor graph with k set at 50. Next, we applied the Leiden algorithm to the k -nearest neighbor graph to cluster cells. Since the number of cell types in the benchmarking datasets is known, we adjusted the Leiden algorithm’s resolution parameter from 0.1 to 3.0 in increments of 0.1 to produce the same number of clusters. We utilized the adjusted Rand index (ARI) and adjusted mutual information (AMI) metrics to evaluate the clustering results.

Data availability

We do not generate any new data in this study. The benchmark datasets used in this study can be accessed at <https://osf.io/hfs2v/>.

Code availability

The source code of SnapATAC2 can be accessed at <https://github.com/kaizhang/SnapATAC2>. The source code for reproducing the benchmarks in this project can be accessed at <https://github.com/kaizhang/single-cell-benchmark>.

Acknowledgements

We thank Y. Zhang, S. Zu and Y.E. Li for valuable discussion and feedback in earlier versions of the manuscript. This work was supported by the Ludwig Institute for Cancer Research (to B.R.), and National Human Genome Research Institute (3U54HG006997-04S2 to B.R.).

Author contributions

Conceptualization: K.Z. and B.R.; Methodology: K.Z.; Software: K.Z.; Investigation: K.Z. and B.R.; Data Curation: K.Z., N.R.Z., and E.J.A.; Writing – Original Draft: K.Z. and B.R.; Writing – Review and Editing: K.Z., N.R.Z., E.J.A. and B.R.; Funding Acquisition: B.R.

Competing interests

B.R. is a co-founder of Epigenome Technologies, and a co-found and consultant of Arima Genomics.

References

1. Preissl, S., Gaulton, K. J. & Ren, B. [Characterizing cis-regulatory elements using single-cell epigenomics](#). *Nature Reviews Genetics* **24**, 21–43 (2022).
2. Lähnemann, D. *et al.* [Eleven grand challenges in single-cell data science](#). *Genome Biology* **21**, (2020).
3. Wolf, F. A., Angerer, P. & Theis, F. J. [SCANPY: Large-scale single-cell gene expression data analysis](#). *Genome Biology* **19**, (2018).
4. Hao, Y. *et al.* [Integrated analysis of multimodal single-cell data](#). *Cell* **184**, 3573–3587.e29 (2021).
5. Granja, J. M. *et al.* [ArchR is a scalable software package for integrative single-cell chromatin accessibility analysis](#). *Nature Genetics* **53**, 403–411 (2021).
6. Stuart, T., Srivastava, A., Madad, S., Lareau, C. A. & Satija, R. [Single-cell chromatin state analysis with signac](#). *Nature Methods* **18**, 1333–1341 (2021).
7. González-Blas, C. B. *et al.* [cisTopic: Cis-regulatory topic modeling on single-cell ATAC-seq data](#). *Nature Methods* **16**, 397–400 (2019).
8. Kim, H.-J. *et al.* [Capturing cell type-specific chromatin compartment patterns by applying topic modeling to single-cell hi-c data](#). *PLOS Computational Biology* **16**, e1008173 (2020).
9. Fang, R. *et al.* [Comprehensive analysis of single cell ATAC-seq data with SnapATAC](#). *Nature Communications* **12**, (2021).
10. Schwartz, G. W. *et al.* [TooManyCells identifies and visualizes relationships of single-cell clades](#). *Nature Methods* **17**, 405–413 (2020).
11. Schwartz, G. W., Zhou, Y., Petrovic, J., Pear, W. S. & Faryabi, R. B. [TooManyPeaks identifies drug-resistant-specific regulatory elements from single-cell leukemic epigenomes](#). *Cell Reports* **36**, 109575 (2021).
12. Haghverdi, L., Büttner, F. & Theis, F. J. [Diffusion maps for high-dimensional single-cell analysis of differentiation data](#). *Bioinformatics* **31**, 2989–2998 (2015).
13. Angerer, P. *et al.* [Destiny: Diffusion maps for large-scale single-cell data in R](#). *Bioinformatics* **32**, 1241–1243 (2015).
14. Gayoso, A. *et al.* [A python library for probabilistic analysis of single-cell omics data](#). *Nature Biotechnology* **40**, 163–166 (2022).
15. Ashuach, T., Reidenbach, D. A., Gayoso, A. & Yosef, N. [PeakVI: A deep generative model for single-cell chromatin accessibility analysis](#). *Cell Reports Methods* **2**, 100182 (2022).
16. Yuan, H. & Kelley, D. R. [scBasset: Sequence-based modeling of single-cell ATAC-seq using convolutional neural networks](#). *Nature Methods* **19**, 1088–1096 (2022).
17. Xiong, L. *et al.* [SCALE method for single-cell ATAC-seq analysis via latent feature extraction](#). *Nature Communications* **10**, (2019).
18. Zhang, R., Zhou, T. & Ma, J. [Multiscale and integrative single-cell hi-c analysis with higashi](#). *Nature Biotechnology* **40**, 254–261 (2021).
19. Maaten, L. van der & Hinton, G. [Visualizing data using t-SNE](#). *Journal of Machine Learning Research* **9**, 2579–2605 (2008).
20. McInnes, L., Healy, J., Saul, N. & Großberger, L. [UMAP: Uniform manifold approximation and projection](#). *Journal of Open Source Software* **3**, 861 (2018).
21. Chari, T. & Pachter, L. [The specious art of single-cell genomics](#). (2021).
22. Chen, H. *et al.* [Assessment of computational methods for the analysis of single-cell ATAC-seq data](#). *Genome Biology* **20**, (2019).

23. Tremblay, N. & Loukas, A. [Approximating spectral clustering via sampling: A review](#). in *Sampling techniques for supervised or unsupervised tasks* 129–183 (Springer International Publishing, 2019).
24. Fowlkes, C., Belongie, S., Chung, F. & Malik, J. [Spectral grouping using the nystrom method](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence* **26**, 214–225 (2004).
25. Lanczos, C. [An iteration method for the solution of the eigenvalue problem of linear differential and integral operators](#). *Journal of Research of the National Bureau of Standards* **45**, 255 (1950).
26. Klabnik, S. & Nichols, C. [The rust programming language, 2nd edition](#). (No Starch Press, 2023).
27. Van Rossum, G. & Drake, F. L. [The python language reference manual](#). (Network Theory Limited, 2011).
28. Virshup, I. *et al.* [The scverse project provides a computational ecosystem for single-cell omics data analysis](#). *Nature Biotechnology* **41**, 604–606 (2023).
29. Belkin, M. & Niyogi, P. [Laplacian eigenmaps for dimensionality reduction and data representation](#). *Neural Computation* **15**, 1373–1396 (2003).
30. Chen, G. [Scalable spectral clustering with cosine similarity](#). in *2018 24th international conference on pattern recognition (ICPR)* (IEEE, 2018).
31. Baek, S. & Lee, I. [Single-cell ATAC sequencing analysis: From data preprocessing to hypothesis generation](#). *Computational and Structural Biotechnology Journal* **18**, 1429–1439 (2020).
32. Zhang, K. *et al.* [A single-cell atlas of chromatin accessibility in the human genome](#). *Cell* **184**, 5985–6001.e19 (2021).
33. Danese, A. *et al.* [EpiScanpy: Integrated single-cell epigenomic analysis](#). *Nature Communications* **12**, (2021).
34. Traag, V. A., Waltman, L. & Eck, N. J. van. [From louvain to leiden: Guaranteeing well-connected communities](#). *Scientific Reports* **9**, (2019).
35. Duò, A., Robinson, M. D. & Soneson, C. [A systematic performance evaluation of clustering methods for single-cell RNA-seq data](#). *F1000Research* **7**, 1141 (2020).
36. Buenrostro, J. D. *et al.* [Integrated single-cell analysis maps the continuous regulatory landscape of human hematopoietic differentiation](#). *Cell* **173**, 1535–1548.e16 (2018).
37. Trevino, A. E. *et al.* [Chromatin and gene-regulatory dynamics of the developing human cerebral cortex at single-cell resolution](#). *Cell* **184**, 5053–5069.e23 (2021).
38. Chen, S., Lake, B. B. & Zhang, K. [High-throughput sequencing of the transcriptome and chromatin accessibility in the same cell](#). *Nature Biotechnology* **37**, 1452–1457 (2019).
39. Ma, S. *et al.* [Chromatin potential identified by shared single-cell profiling of RNA and chromatin](#). *Cell* **183**, 1103–1116.e20 (2020).
40. Yao, Z. *et al.* [A transcriptomic and epigenomic cell atlas of the mouse primary motor cortex](#). *Nature* **598**, 103–110 (2021).
41. Luecken, M. *et al.* [A sandbox for prediction and integration of DNA, RNA, and proteins in single cells](#). in *Proceedings of the neural information processing systems track on datasets and benchmarks* (eds. Vanschoren, J. & Yeung, S.) vol. 1 (Curran, 2021).
42. Zemke, N. R. *et al.* [Comparative single cell epigenomic analysis of gene regulatory programs in the rodent and primate neocortex](#). (2023).
43. Lee, D.-S. *et al.* [Simultaneous profiling of 3D genome structure and DNA methylation in single human cells](#). *Nature Methods* **16**, 999–1006 (2019).
44. Zhou, J. *et al.* [Robust single-cell hi-c clustering by convolution- and random-walk-based imputation](#). *Proceedings of the National Academy of Sciences* **116**, 14011–14018 (2019).
45. Raimundo, F., Vallot, C. & Vert, J.-P. [Tuning parameters of dimensionality reduction methods for single-cell RNA-seq analysis](#). *Genome Biology* **21**, (2020).
46. Ruf-Zamojski, F. *et al.* [Single nucleus multi-omics regulatory landscape of the murine pituitary](#). *Nature Communications* **12**, (2021).
47. Zhu, C. *et al.* [Joint profiling of histone modifications and transcriptome in single cells from mouse brain](#). *Nature Methods* **18**, 283–292 (2021).

- 589 48. Kumar, A., Rai, P. & Daumé, H. Co-regularized multi-view spectral clustering. in *Proceedings of the 24th inter-*
590 *national conference on neural information processing systems* 1413–1421 (Curran Associates Inc., 2011).
- 591 49. Li, M., Lian, X.-C., Kwok, J. T. & Lu, B.-L. [Time and space efficient spectral clustering via column sampling](#).
592 in *CVPR 2011* (IEEE, 2011).
- 593 50. Stoeckius, M. *et al.* [Simultaneous epitope and transcriptome measurement in single cells](#). *Nature Methods* **14**,
594 865–868 (2017).
- 595 51. Chen, G. [A scalable spectral clustering algorithm based on landmark-embedding and cosine similarity](#). in
596 *Lecture notes in computer science* 52–62 (Springer International Publishing, 2018).
- 597 52. Ng, A., Jordan, M. & Weiss, Y. [On spectral clustering: Analysis and an algorithm](#). in *Advances in neural*
598 *information processing systems* (eds. Dietterich, T., Becker, S. & Ghahramani, Z.) vol. 14 (MIT Press, 2001).

Supplementary Note 1: Supplementary material

Extended Data Table 1 | Comparing the Runtime and memory usage of different dimensionality reduction methods.

Extended Data Table 2 | Comparing the runtime and memory usage of the preprocessing components in ArchR and SnapATAC2.

Extended Data Table 3 | Benchmarking the cell embedding methods on scATAC-seq datasets.

Extended Data Table 4 | Benchmarking the cell embedding methods on scHi-C datasets.

Extended Data Table 5 | Benchmarking the cell embedding methods on scRNA-seq datasets.

Extended Data Table 6 | Comparing the cell embeddings generated by the Nystrom method with those produced using the subsampled LSI method.

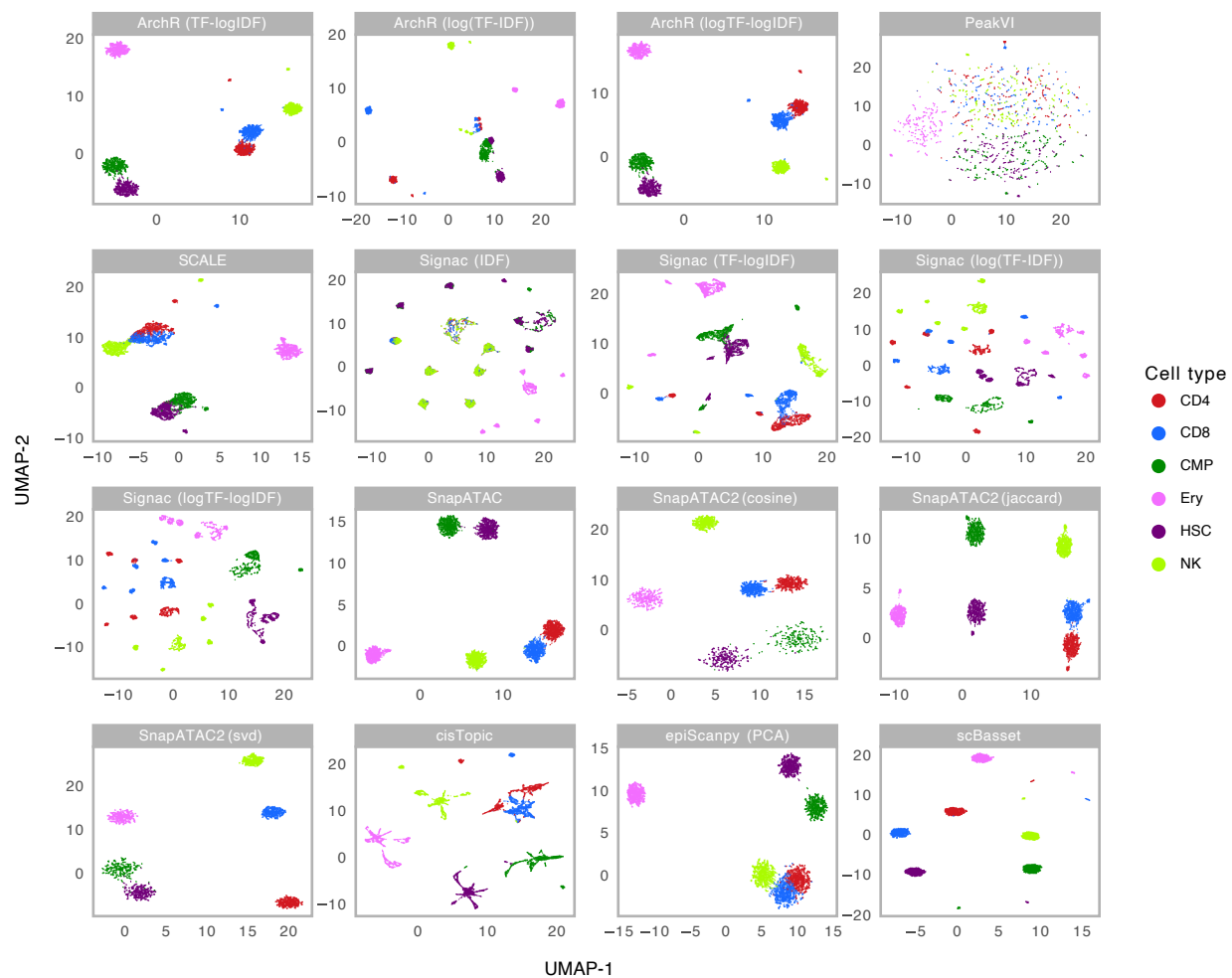


Figure S1 | Cell embeddings for the Chen *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Chen *et al.* Cells are color-coded by cell type labels. ArchR (TF-logIDF): the embeddings produced by ArchR using the “TF-logIDF” method. ArchR (log(TF-IDF)): the embeddings produced by ArchR using the “log(TF-IDF)” method. This is the default method used by ArchR and was reported in the main text. ArchR (logTF-logIDF): the embeddings produced by ArchR using the “logTF-logIDF” method. PeakVI: the embeddings produced by PeakVI using the variational autoencoder model. SCALE: the embeddings produced by SCALE. Signac (IDF): the embeddings produced by Signac using the “IDF” method. Signac (TF-logIDF): the embeddings produced by Signac using the “TF-logIDF” method. Signac (log(TF-IDF)): the embeddings produced by Signac using the “log(TF-IDF)” method. This is the default method used by Signac and was reported in the main text. Signac (logTF-logIDF): the embeddings produced by Signac using the “logTF-logIDF” method. SnapATAC: the embeddings produced by SnapATAC using the spectral embedding with Jaccard index as the similarity measure. SnapATAC2 (cosine): the embeddings produced by SnapATAC2 using the spectral embedding with cosine similarity as the similarity measure. SnapATAC2 (Jaccard): the embeddings produced by SnapATAC2 using the spectral embedding with Jaccard index as the similarity measure. SnapATAC2 (svd): Same as “SnapATAC2 (cosine)” except that the diagonal of the similarity matrix is not set to zero. cisTopic: the embeddings produced by cisTopic using the latent Dirichlet allocation model. epiScanpy: the embeddings produced by EpiScanpy using the principal component analysis. scBasset: the embeddings produced by scBasset using the convolutional neural network model.

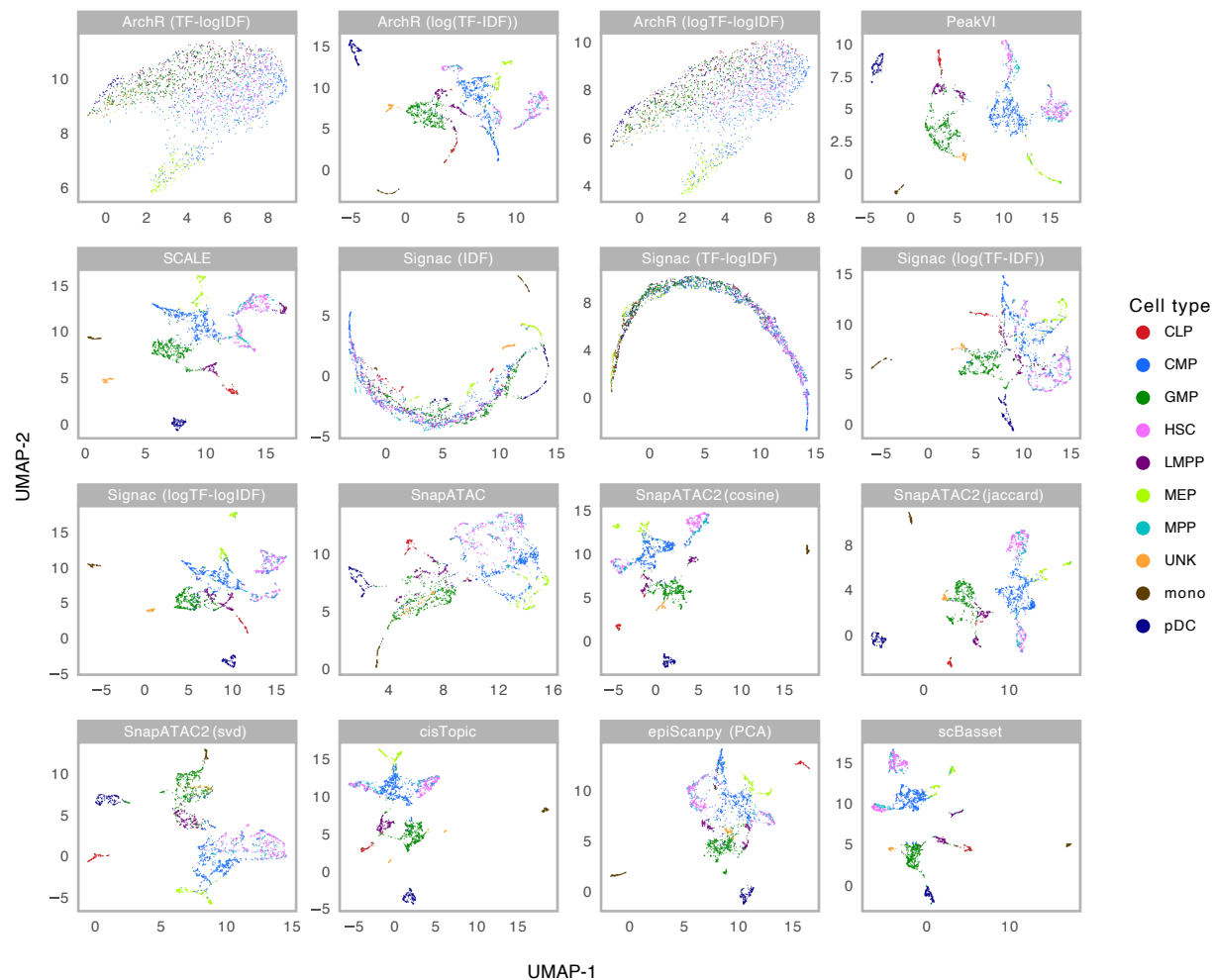


Figure S2 | Cell embeddings for the Buenrostro *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Buenrostro *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

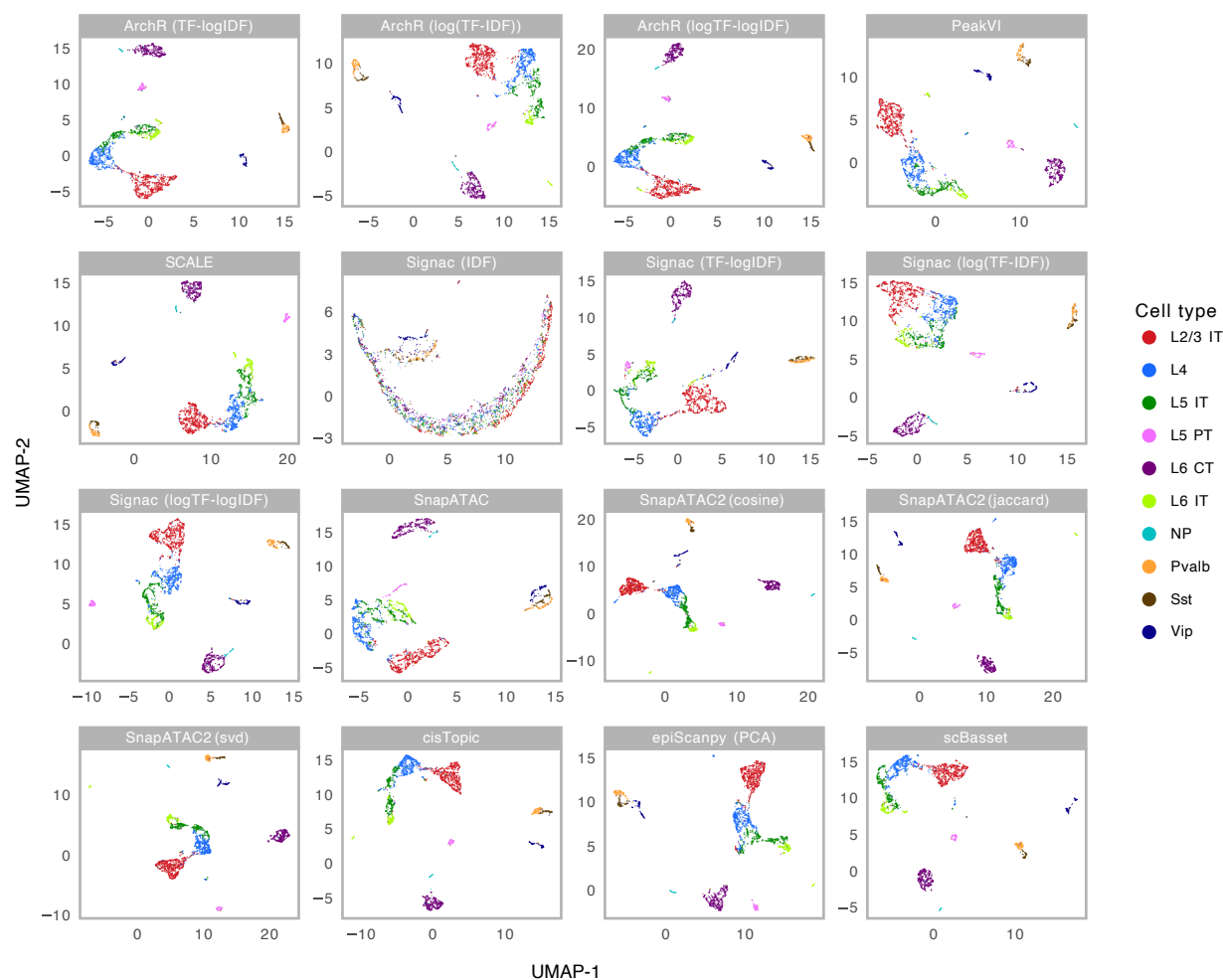


Figure S3 | Cell embeddings for the 10x Brain 5k dataset. UMAP visualization of different cell embedding methods on 10x Brain 5k dataset. Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

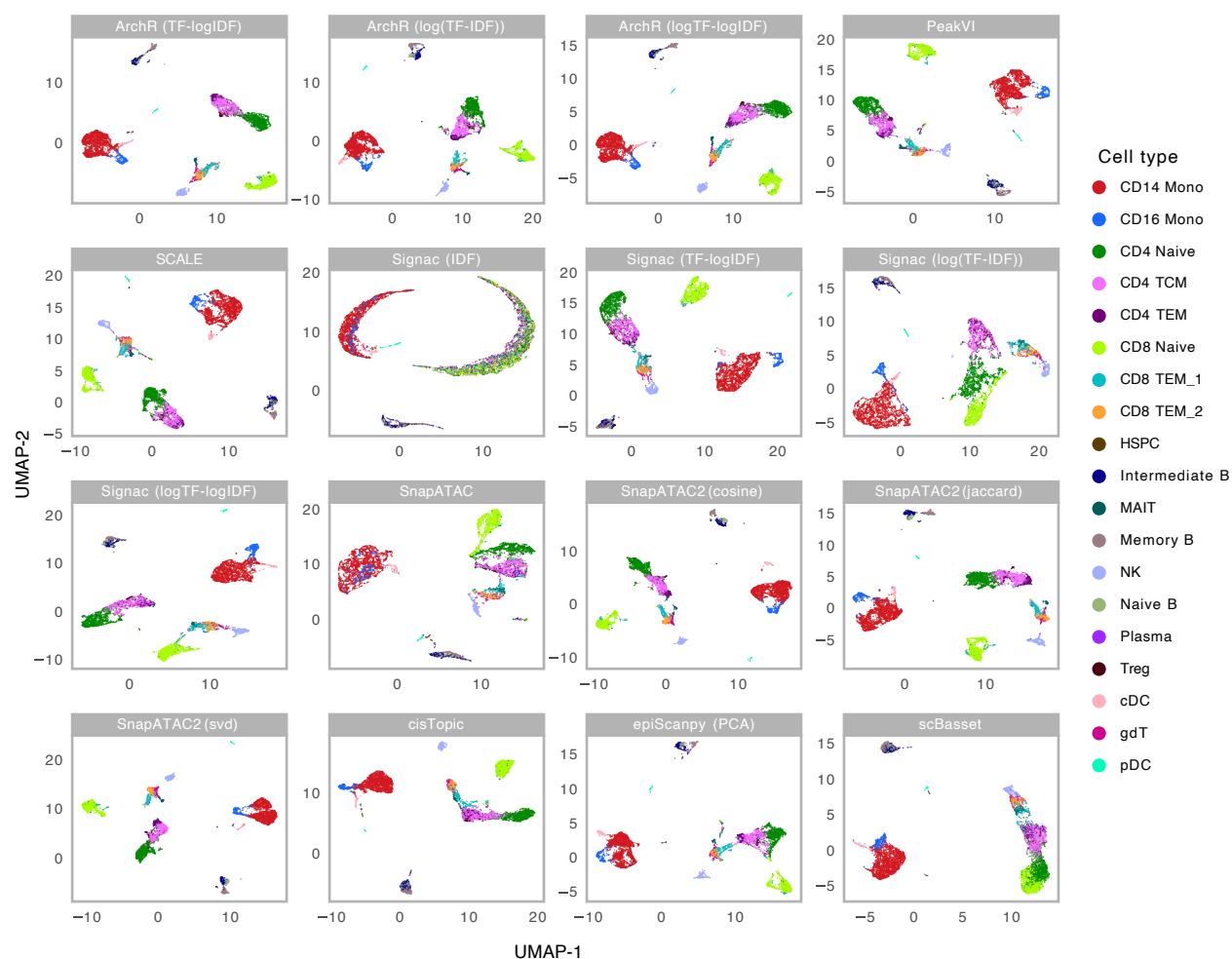


Figure S4 | Cell embeddings for the 10x PBMC 10k dataset. UMAP visualization of different cell embedding methods on 10x PBMC 10k dataset. Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

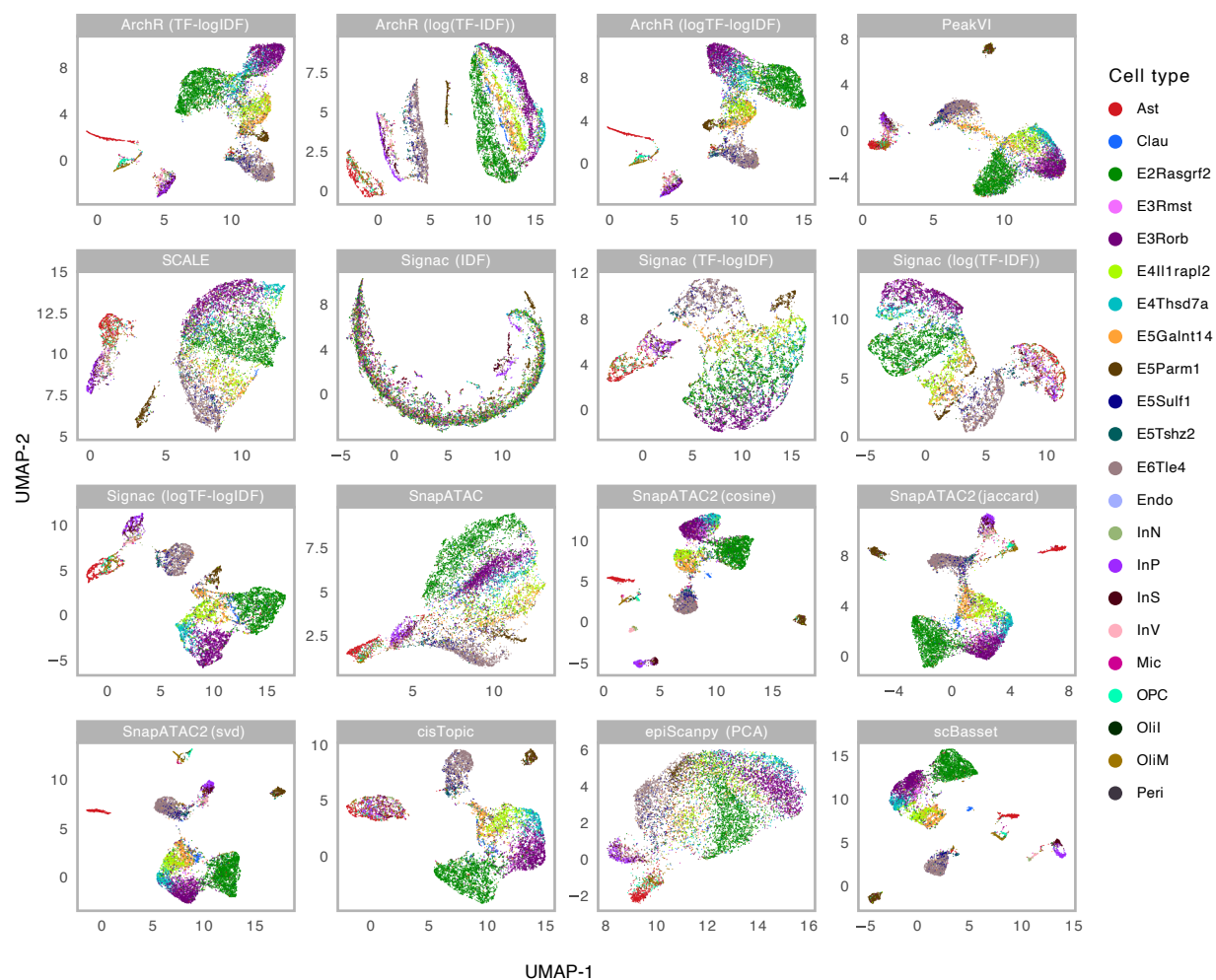


Figure S5 | Cell embeddings for the Chen *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Chen *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

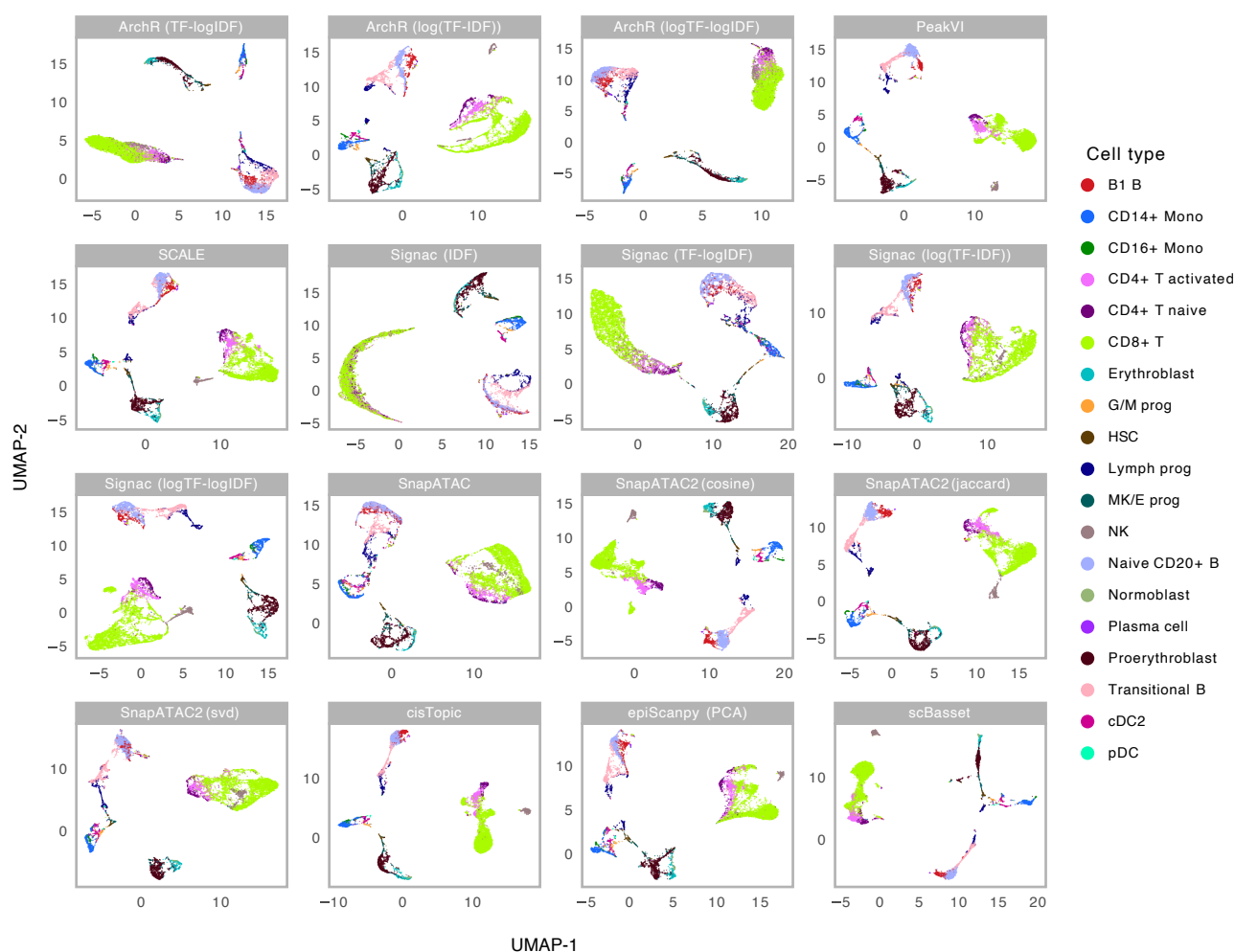


Figure S6 | Cell embeddings for the GSE194122 dataset. UMAP visualization of different cell embedding methods on the dataset from GSE194122. Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

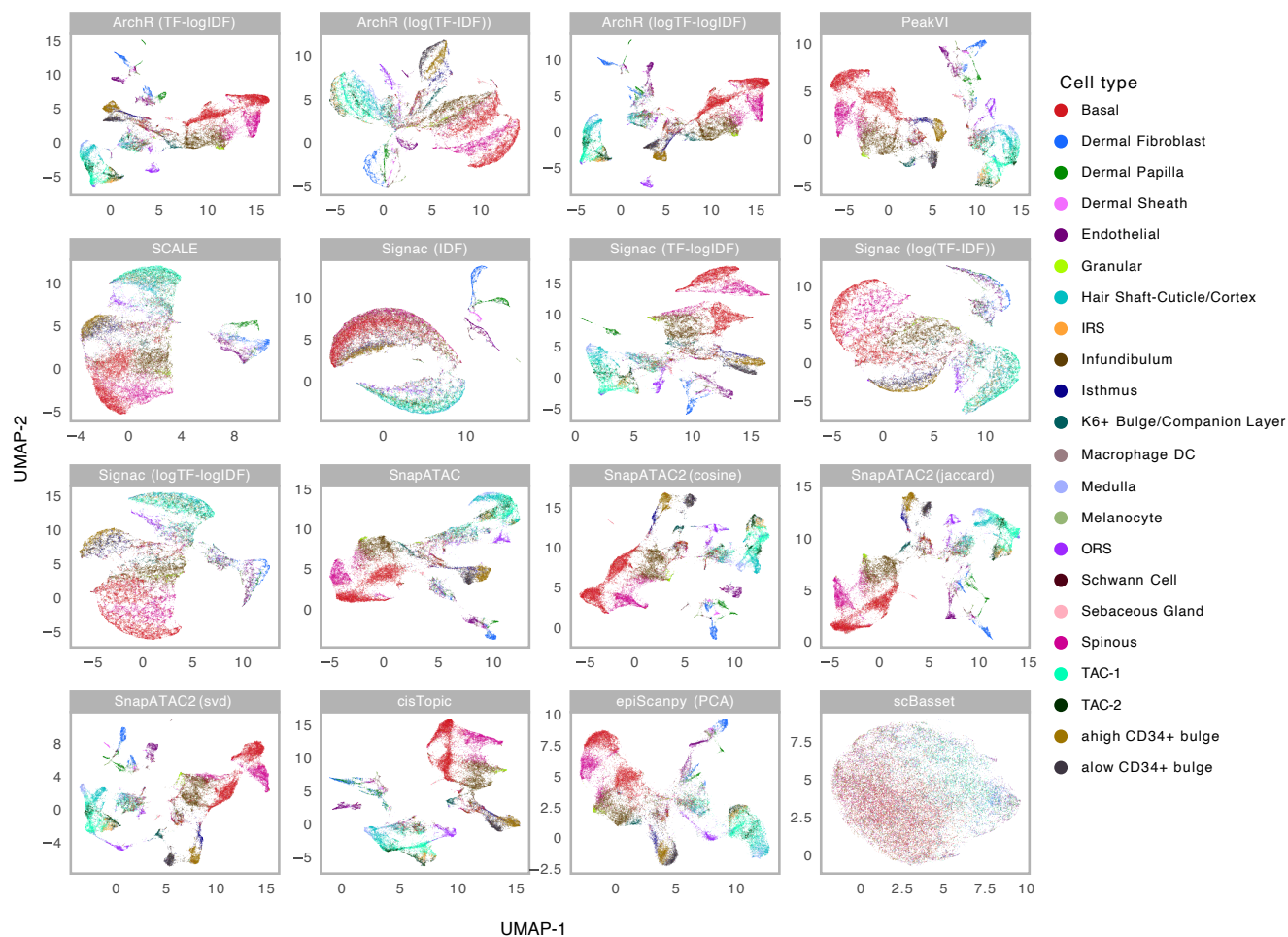


Figure S7 | Cell embeddings for the Ma *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Ma *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

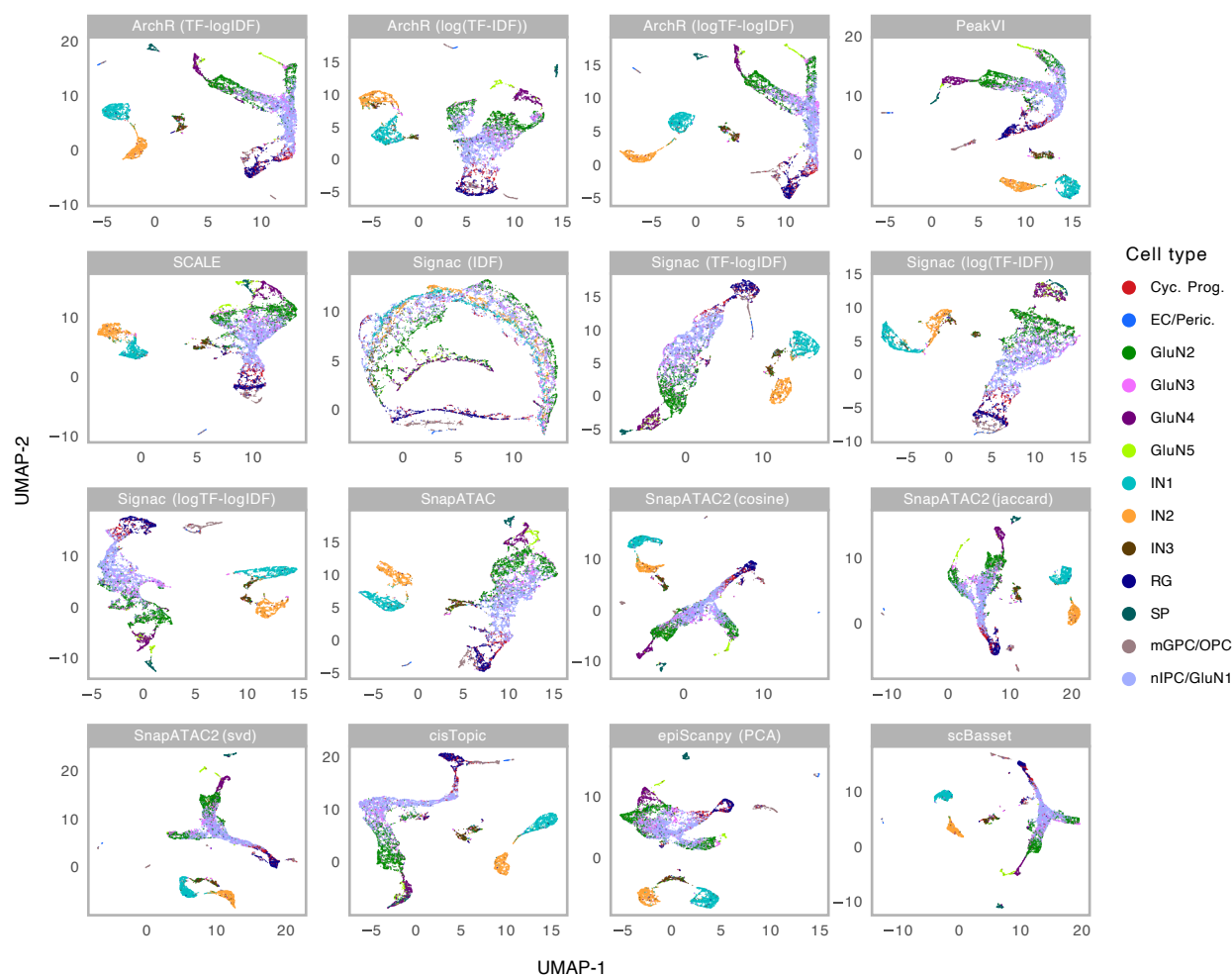


Figure S8 | Cell embeddings for the Trevino *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Trevino *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

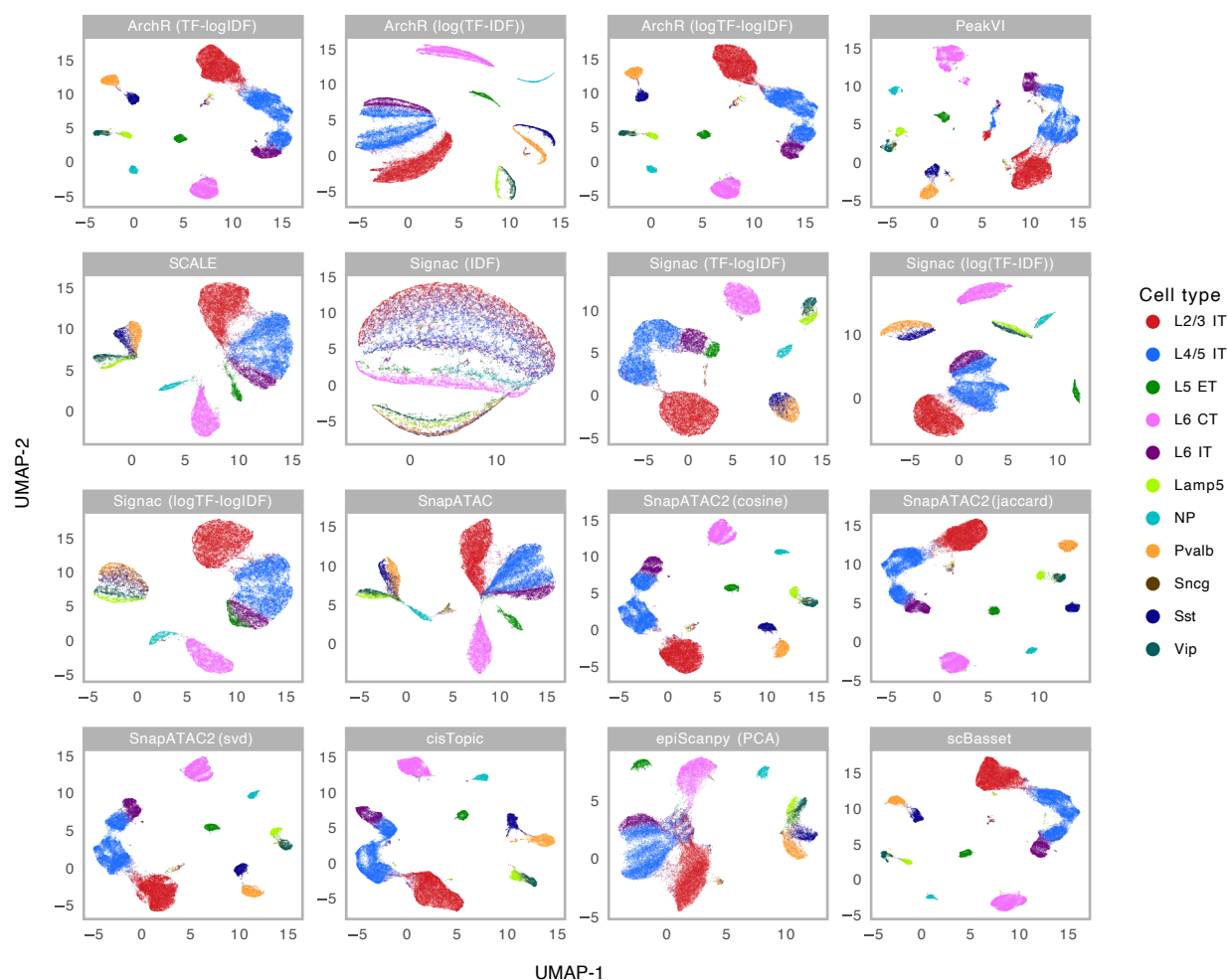


Figure S9 | Cell embeddings for the Yao *et al.* dataset. UMAP visualization of different cell embedding methods on the dataset from Yao *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

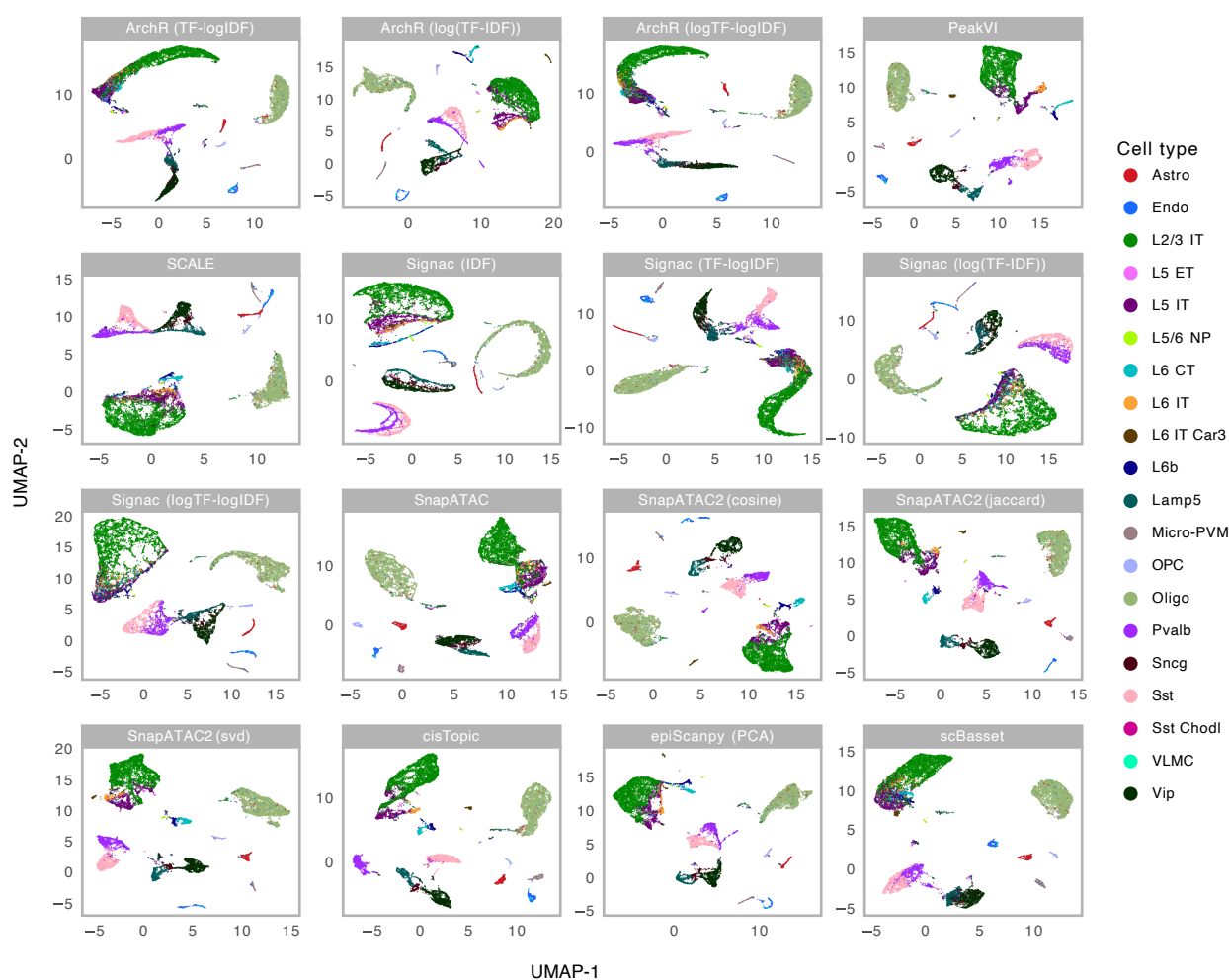


Figure S10 | Cell embeddings for the Zemke *et al.* human dataset. UMAP visualization of different cell embedding methods on the human dataset from Zemke *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

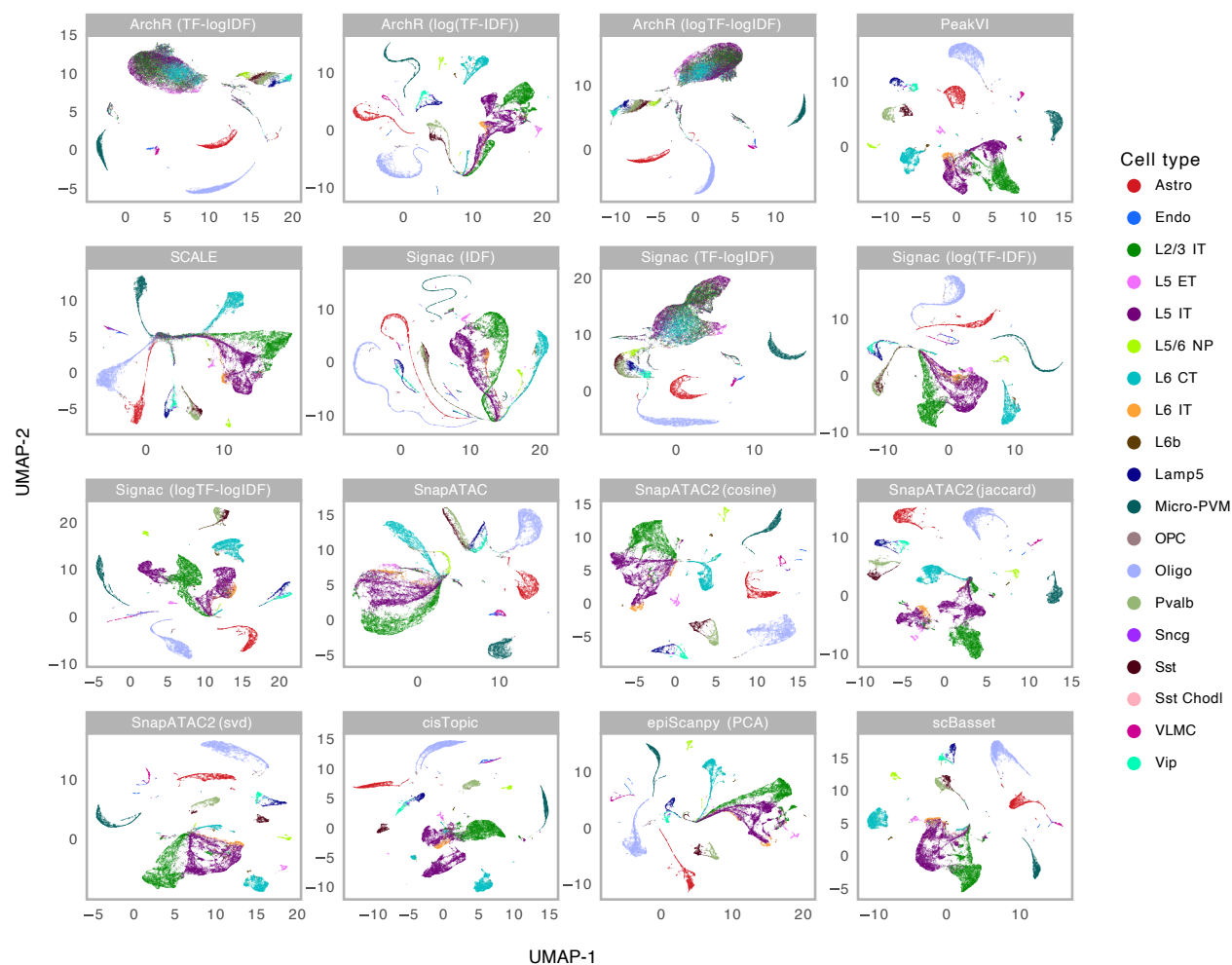


Figure S11 | Cell embeddings for the Zemke *et al.* mouse dataset. UMAP visualization of different cell embedding methods on the mouse dataset from Zemke *et al.* Cells are color-coded by cell type labels. See the caption of **Fig. S1** for the description of the methods.

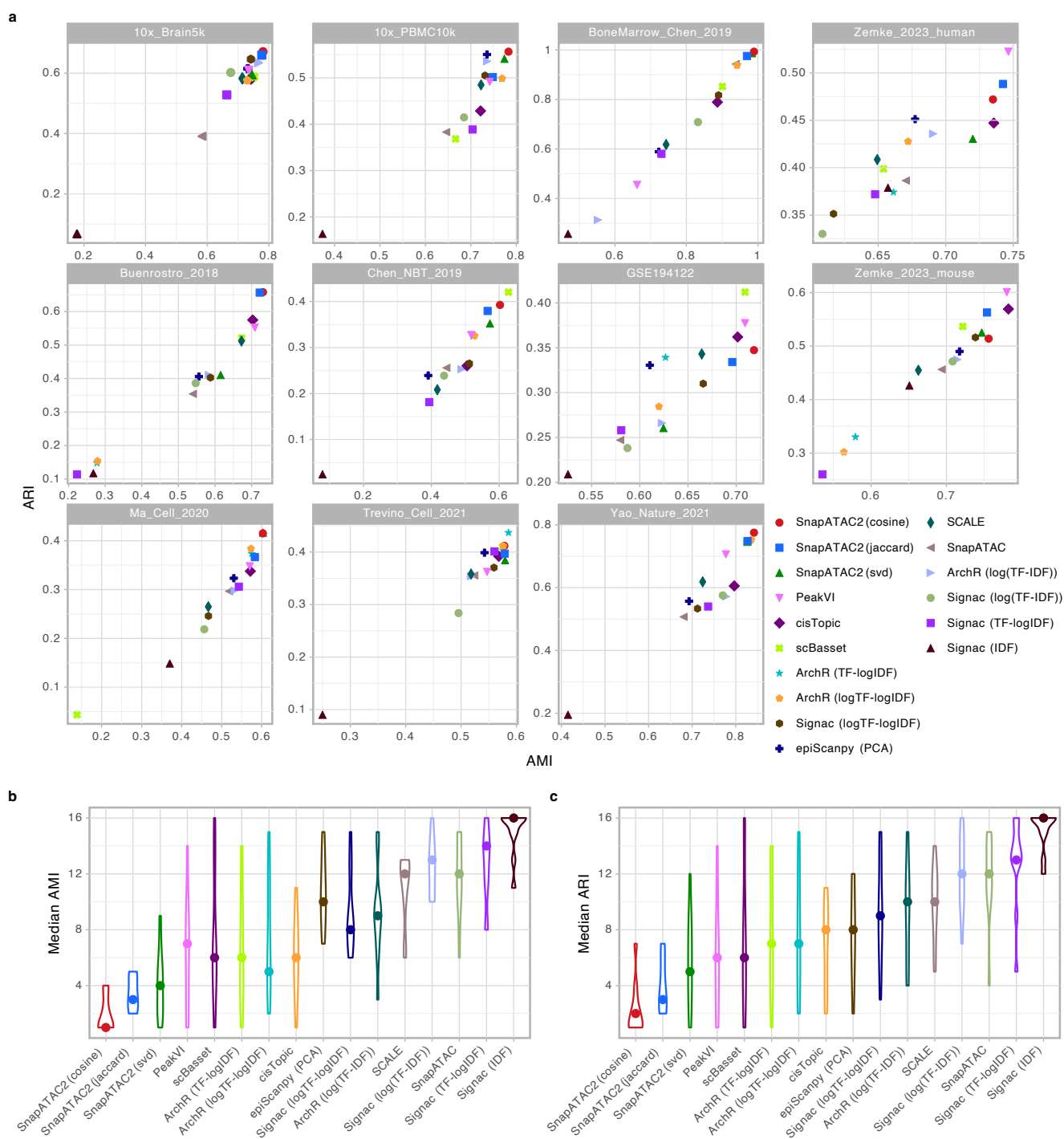


Figure S12 | Summary of benchmarking results of various dimensionality reduction methods on scATAC-seq datasets. a, Scatter plots showing the relationship between the adjusted mutual information (AMI) and the adjusted Rand index (ARI) for the cell embeddings produced by different methods. The AMI and ARI are computed between the cell type labels and the clusters identified by each method. See the caption of **Fig. S1** for the description of the methods. **b,c,** Violin plots showing the distribution of AMI (**b**) and ARI (**c**) scores for each method across all eleven scATAC-seq benchmark datasets. The median rank of each method is indicated by the dot in the violin plot.

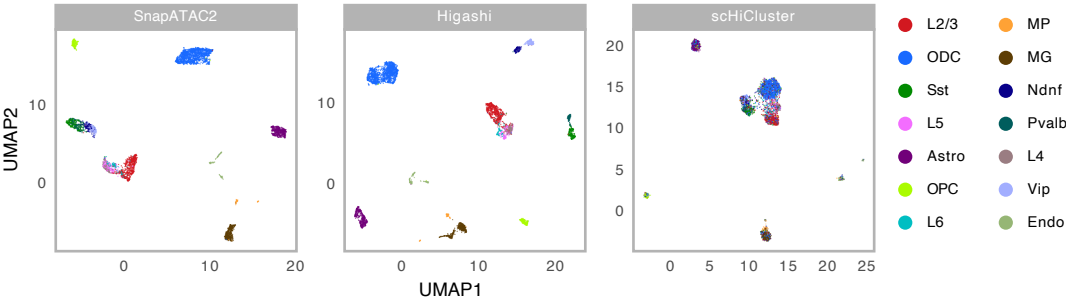


Figure S13 | UMAP visualization of the embeddings produced by SnapATAC2, Higashi, and scHiCluster on single-cell Hi-C dataset from Lee et al.

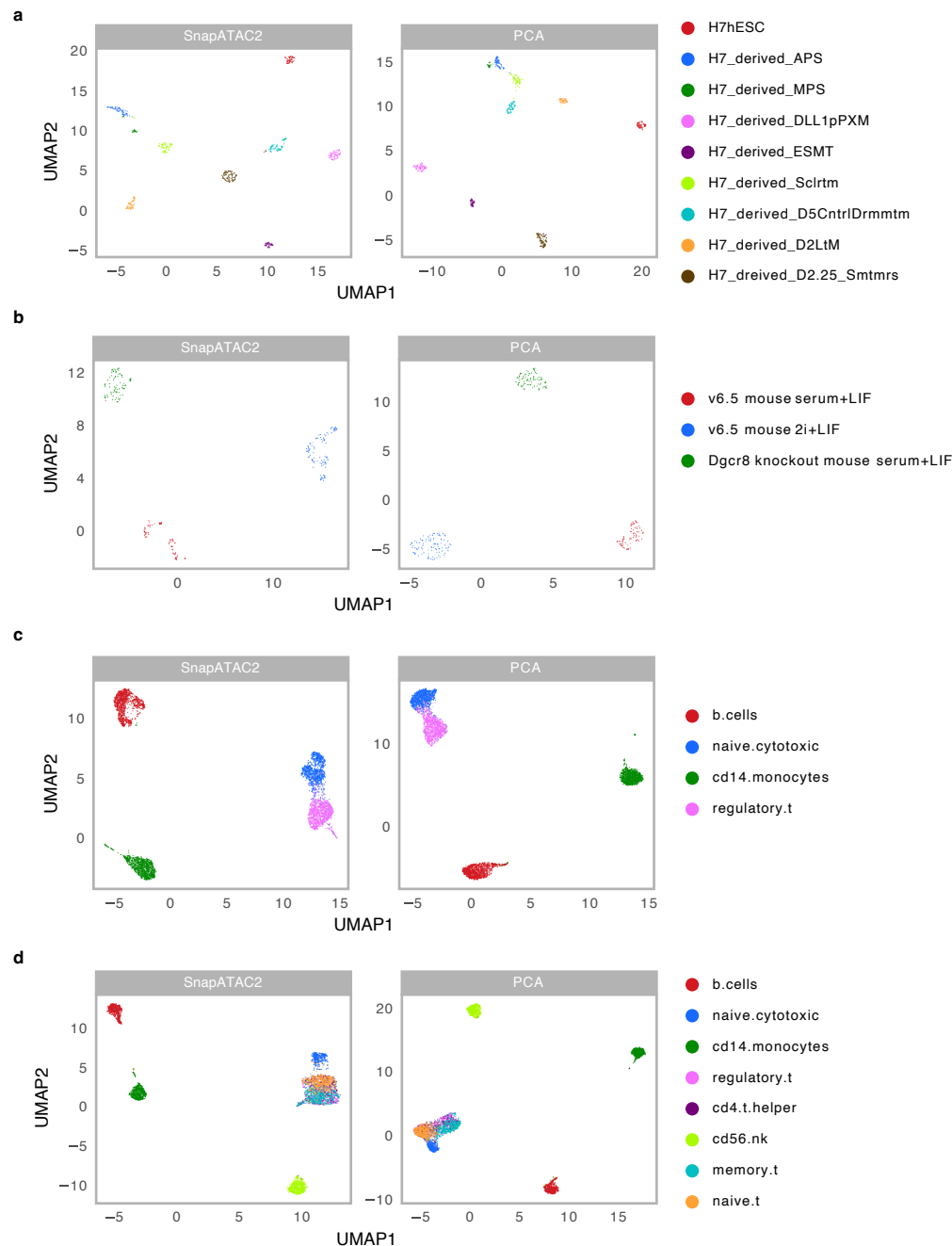


Figure S14 | UMAP visualization of the embeddings produced by SnapATAC2 and PCA (scanpy) on various single cell RNA-seq datasets. The datasets include Koh (a), Kumar (b), Zhengmix4eq (c), Zhengmix8eq (d).

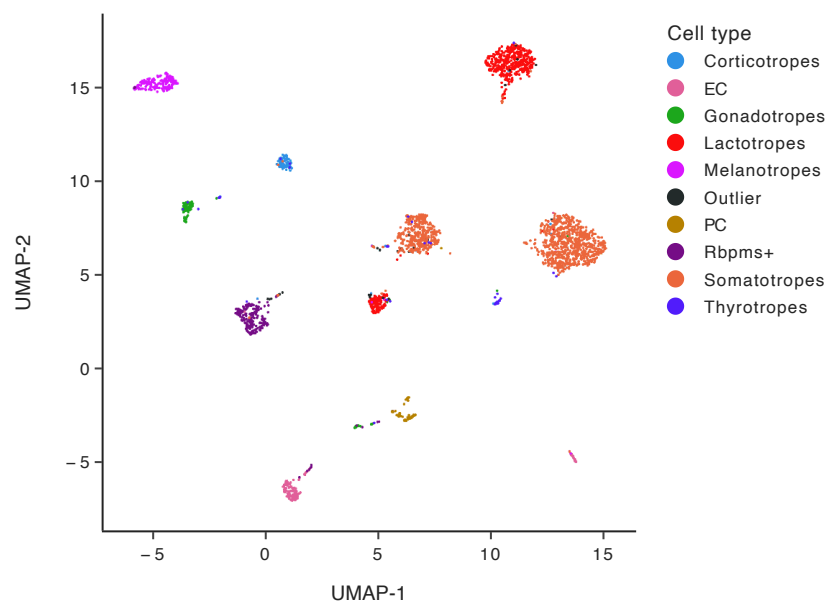


Figure S15 | SnapATAC2 unveils fine-grained cellular heterogeneity in single-cell DNA methylation data from Ruf-Zamojski *et al.*

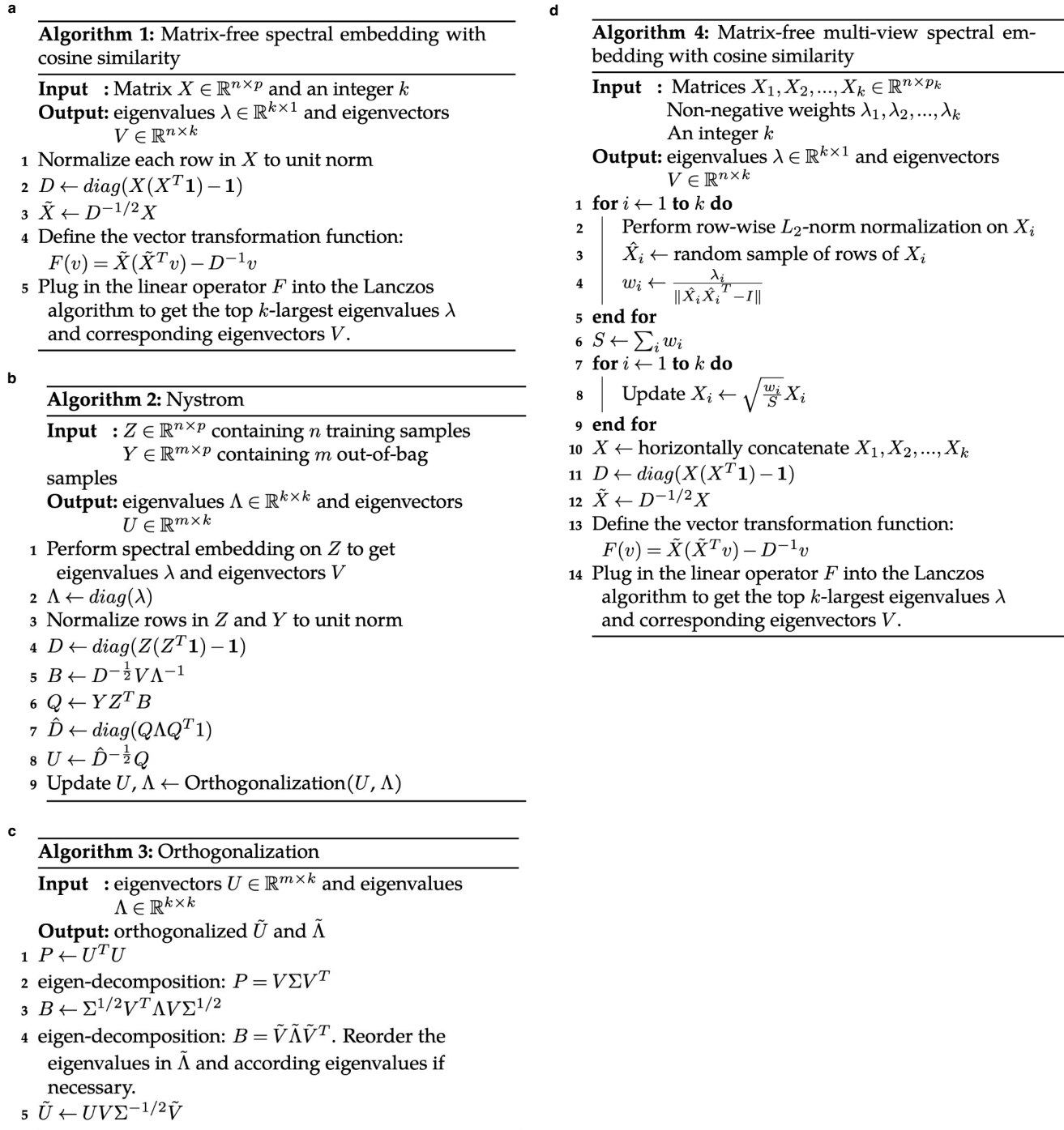


Figure S16 | The pseudocodes of various algorithms used in this study. **a**, The pseudocode of the matrix-free spectral embedding algorithm. **b**, The pseudocode of the Nyström algorithm for performing the out-of-sample embedding. **c**, The pseudocode for performing orthogonalization on the eigenvectors produced by the Nyström algorithm. **d**, The pseudocode of the matrix-free multi-view spectral embedding algorithm.